

Prototipo de sistema de traducción de Lengua de Señas Argentina con procesamiento en tiempo cuasi real

Maite Nigro Francisco Veron

Director: Esp. Ing. Gonzalo Avalos Ribas

Co-director: Ing. Bruno Constanzo

Universidad Nacional de Mar del Plata

Facultad de Ingeniería

Departamento de Ingeniería en Informática

Marzo 2026

Agradecimientos

De Maite: Para la Flaqui.

De Francisco: Para el Diego y Baco.

A Miriam Rolls, por su orientación sobre la LSA y la validación del vocabulario del conjunto de datos.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	1
1.3. Problemática	2
1.4. Alcance y limitaciones	2
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Marco teórico, estado del arte y metodología	5
3.1. Marco teórico	5
3.1.1. Arquitectura híbrida de partición Cliente-Nube (Edge-Cloud)	5
3.1.2. Estructura lingüística de la LSA y traducción Topic-Comment . . .	5
3.1.3. Visión computacional y reconocimiento basado en esqueletos	6
3.1.4. Arquitectura neuronal temporal: TinySkeletonClassifier	6
3.1.5. Modelos de lenguaje pequeños (SLM) y adaptación LoRA	6
3.1.6. Animación y representación visual mediante Avatar 3D	7
3.2. Estado del arte	7
3.2.1. Desarrollos en Argentina y la región	7
3.2.2. Sistemas internacionales de reconocimiento y síntesis	7
3.3. Metodología de desarrollo	8
3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial . .	8
3.4. Herramientas y tecnologías empleadas	10
4. Análisis de requerimientos	11
4.1. Requerimientos funcionales	11
4.2. Requerimientos no funcionales	12
4.3. Restricciones	12
4.3.1. Especificación del entorno de ejecución cliente	13
4.4. Casos de uso	15
4.4.1. Casos de uso esenciales en formato breve	15
4.4.2. Casos de uso expandidos	16
5. Diseño e Implementación de la Arquitectura del Sistema	18
5.1. Arquitectura global y partición Cliente-Nube	18
5.1.1. Justificación de la partición y modelo de privacidad	18
5.1.2. Flujo integral de datos entre componentes.	19
5.1.3. Esquema de comunicación y contrato de interfaces (API REST) . .	23

5.2.	Subsistema de reconocimiento gestual (Clasificador)	26
5.2.1.	Captura e ingesta de video con MediaPipe Holistic	26
5.2.2.	Normalización geométrica y corrección de distorsión de relación de aspecto	26
5.2.3.	Preprocesamiento: recorte temporal, submuestreo e interpolación . .	26
5.2.4.	Arquitectura neuronal TinySkeletonClassifier (Conv1D + transfor- mer + attention pooling)	26
5.2.5.	Segmentación continua y políticas anti-rebote (RepeatGate)	26
5.3.	Subsistema de traducción semántica bidireccional	26
5.3.1.	Canal directo (LSA $\rightarrow$ Español): fine-tuning LoRA y loss masking .	26
5.3.2.	Canal inverso (Español $\rightarrow$ LSA): estructuración gramatical topic- comment	28
5.3.3.	Optimización de inferencia: formato GGUF y cuantización a 4 bits .	30
5.3.4.	Manejo de contexto y memoria conversacional deslizante	31
5.4.	Integración en Google Meet y extensión de navegador	32
5.4.1.	Desafíos del entorno Chrome Manifest V3: aislamiento offscreen y sandbox	32
5.4.2.	Intercepción de Media y canvas.captureStream()	33
5.4.3.	Motor Local ILSA (FastAPI)	35
5.5.	Representación gestual Inversa mediante avatar 3D	36
5.5.1.	Controlador de animaciones esqueléticas por glosas	36
5.5.2.	Mecanismo de contingencia: deletreo manual dactilológico	36
5.6.	Diseño conceptual de la arquitectura escalable en la nube	36
5.6.1.	Especificación para entornos de producción concurrente	37
5.6.2.	Instancia de validación del prototipo mediante túnel seguro	38
6.	Experimentación, Resultados y Discusión	40
6.1.	Corpus y metodología de adquisición de Datos de LSA	41
6.1.1.	Definición y validación del vocabulario de 100 clases	41
6.1.2.	Protocolo de grabación y curación del dataset propio	41
6.2.	Evaluación empírica del clasificador gestual	41
6.2.1.	Métricas offline vs. rendimiento en cámara real	41
6.2.2.	Impacto de la resolución temporal: 16 vs. 32 fotogramas	41
6.2.3.	Búsqueda automatizada de hiperparámetros con Optuna y Análisis de sobreajuste	41
6.2.4.	Diagnóstico de confusiones mediante feature engineering y separabi- lidad	41
6.2.5.	Impacto cuantitativo de la corrección de aspect ratio	41
6.2.6.	Desempeño final del clasificador baseline en vivo	41
6.3.	Evaluación de los módulos de traducción semántica	41
6.3.1.	Benchmark comparativo de modelos SLM (Familias Qwen2.5, Llama 3.2 y SmolLM2)	41
6.3.2.	Evaluación cuantitativa: BLEU, ROUGE-L, METEOR y exactitud	41
6.3.3.	Análisis de simetría en el aprendizaje: traducción directa vs. estruc- turación inversa	41
6.3.4.	Compromiso entre calidad lingüística y latencia de inferencia	41
6.4.	Evaluación del sistema integrado y rendimiento en tiempo real	41
6.4.1.	Validación funcional de extremo a extremo en videollamadas reales	41

6.4.2.	Análisis de latencias totales por etapa del pipeline	41
6.4.3.	Consumo de recursos de hardware: máquina de desarrollo vs. máquina objetivo	41
6.4.4.	Discusión técnica y lecciones de ingeniería aprendidas	41
7.	Conclusiones y trabajo futuro	42
7.1.	Conclusiones	42
7.1.1.	Cumplimiento de objetivos del proyecto	42
7.1.2.	Aportes técnicos y de accesibilidad	42
7.1.3.	Impacto social en el marco de la Ley 27.710	42
7.2.	Trabajo futuro	42
7.2.1.	Implementación física del clúster semántico escalable	42
7.2.2.	Expansión del corpus multiseñante y mitigación de variabilidad	42
7.2.3.	Incorporación de rasgos no manuales (expresiones faciales)	42
7.2.4.	Generalización a nuevas plataformas de comunicación virtual	42

Resumen

En la actualidad, la interacción virtual entre personas sordas y oyentes se limita casi exclusivamente a la comunicación escrita o a la transcripción unilateral de voz a texto, lo que genera una marcada asimetría en la comunicación expresiva durante videollamadas. Este Trabajo Final de Grado desarrolla un prototipo de traducción bidireccional entre la Lengua de Señas Argentina (LSA) y el español oral estructurado bajo una arquitectura híbrida de computación en el borde y servicios en la nube con procesamiento en tiempo pseudo-real.

El sistema articula dos canales comunicacionales complementarios. En el sentido señante-oyente, la computadora del usuario captura localmente el flujo de video desde la cámara web, extrae coordenadas articulares tridimensionales mediante visión computacional y clasifica un vocabulario de 100 señas mediante una red neuronal ligera (Conv1D + Transformer con *attention pooling*). Con el fin de evitar la saturación de memoria y cómputo que provocaría ejecutar modelos de lenguaje generativos en dispositivos convencionales, las glosas clasificadas son enviadas hacia un backend semántico en la nube que transforma la secuencia en oraciones fluidas en español rioplatense, las cuales retornan al cliente para su proyección en subtítulos o síntesis de voz. En sentido inverso (oyente-señante), el sistema procesa la voz del interlocutor oyente mediante un módulo semántico inverso en la nube que estructura las glosas, enviándolas a un avatar tridimensional en el cliente que ejecuta las animaciones correspondientes o recurre al deletreo dactilológico en términos fuera del léxico.

El módulo clasificador gestual local alcanza un 98,28 % de exactitud top-1 en validación fuera de línea y un 77,3 % de acierto top-1 (94,8 % top-3) en cámara real. Por su parte, los modelos de lenguaje evaluados (Qwen2.5, Llama 3.2 y SmolLM adaptados mediante LoRA y cuantizados a GGUF Q4_K_M) superan el 94 % de exactitud y alcanzan latencias de inferencia de entre 200 y 450 ms. Como mecanismo de integración para videollamadas, el sistema se acopla a Google Meet mediante una extensión de navegador conectada al motor local ILSA, asegurando la privacidad del usuario al no transmitir video crudo fuera del dispositivo cliente. Finalmente, se deja documentada la propuesta de diseño de una arquitectura en la nube escalable para soportar la demanda concurrente en entornos de producción.

Palabras clave:

Lengua de Señas Argentina, reconocimiento de gestos, arquitectura híbrida cliente-nube, traducción semántica bidireccional, avatar 3D, accesibilidad.

Capítulo 1

Introducción

1.1. Contexto

Tradicionalmente, la sociedad abordó a la comunidad sorda desde una perspectiva clínica centrada en la rehabilitación y la corrección auditiva. Ese enfoque omite que la comunidad sorda se identifica, ante todo, como una minoría lingüística con lengua, cultura y gramática propias, independientes del español oral. En la Argentina, la LSA cuenta con reconocimiento legal formal a través de la Ley 27.710 (República Argentina, 2020), la cual garantiza su preservación y el derecho de las personas sordas a comunicarse en su lengua natural en todos los ámbitos sociales.

En el contexto digital contemporáneo, las plataformas de videollamada se han convertido en herramientas centrales para el ámbito laboral, educativo y la atención de situaciones críticas. Sin embargo, las soluciones tecnológicas actuales no ofrecen una arquitectura bilingüe integrada que admita la LSA como un flujo comunicacional válido en tiempo pseudo-real. Los recursos existentes suelen restringirse a diccionarios estáticos o repositorios de video desvinculados, carentes de capacidad de interpretación contextual interactiva (INJU, 2024).

1.2. Motivación

Este proyecto nace de la necesidad de devolver autonomía funcional a las personas señantes en interacciones cotidianas y de emergencia mediante herramientas informáticas. Resulta indispensable pasar de un paradigma asistencialista a uno de integración lingüística, en el que la tecnología actúe como un puente transparente y bidireccional. La posibilidad de mitigar la dependencia de intérpretes humanos presenciales en situaciones imprevistas, garantizar la estricta privacidad de los datos personales y evitar los costos asociados a intermediarios impulsa una solución asistida por inteligencia artificial.

Asimismo, desde la perspectiva de la ingeniería de software, surge la motivación de concebir una arquitectura viable: ejecutar la captura visual y el reconocimiento preliminar en el dispositivo del usuario para proteger su intimidad, delegando la pesada inferencia semántica a servicios remotos para democratizar el acceso en computadoras de recursos limitados.

1.3. Problemática

La problemática central radica en la asimetría funcional de las plataformas de comunicación virtual: mientras que la accesibilidad receptiva (de voz a texto) está ampliamente resuelta mediante subtítulo automático, la comunicación expresiva para usuarios sordos permanece desatendida. Esta carencia fuerza al usuario a recurrir al texto escrito, el cual opera para muchos miembros de la comunidad sorda como una segunda lengua.

A esta brecha se añade la vacancia tecnológica local: los modelos de visión y lenguaje existentes en el estado del arte internacional están sesgados hacia la Lengua de Señas Americana (ASL), resultando inaplicables a la LSA debido a diferencias léxicas, dactilológicas y sintácticas sustanciales. Esta disparidad hace imperativa la construcción de un corpus propio y de modelos específicamente adaptados al contexto argentino.

Por último, emerge una problemática de rendimiento de hardware: la ejecución simultánea de videoconferencias, extracción de mallas corporales densas, clasificadores neuronales y modelos de lenguaje de gran escala (LLM) satura de manera crítica las computadoras estándar de los usuarios finales (procesadores móviles de 4 núcleos y 8 GB de RAM), provocando caídas severas de cuadros y congelamientos que invalidan la experiencia en tiempo real.

1.4. Alcance y limitaciones

El sistema desarrollado comprende un prototipo funcional de traducción bidireccional en tiempo pseudo-real integrado para videollamadas. El canal directo procesa las señas captadas por la cámara web en el cliente, clasifica las glosas correspondientes y recurre a un módulo semántico en la nube para sintetizar oraciones fluidas en español mediante texto y audio para el oyente. El canal inverso transcribe la voz del oyente, traduce en la nube la estructura del español a secuencias sintácticas de LSA y comanda un avatar tridimensional en el cliente que reproduce los movimientos gestuales o ejecuta deletreo manual dactilológico en caso de vocablos no contemplados en el léxico base.

El alcance funcional está delimitado a un vocabulario representativo de 100 señas de LSA seleccionadas y validadas junto a una profesional experta en la lengua (Miriam Rolls), orientadas a situaciones de emergencia y vida cotidiana.

En lo que respecta a la infraestructura de despliegue, el alcance comprende la especificación y diseño formal de una arquitectura escalable en la nube para el desacoplamiento semántico. Para la validación empírica del prototipo se implementa una instancia de referencia remota comunicada mediante túneles seguros, delimitando la puesta en producción y el aprovisionamiento de clústeres elásticos con auto-escalado fuera de los recursos financieros del presente Trabajo Final de Grado.

Capítulo 2

Objetivos

2.1. Objetivo general

Desarrollar un prototipo de sistema de traducción bidireccional entre la LSA y el español rioplatense con procesamiento en tiempo pseudo-real, diseñado para operar sobre plataformas de videoconferencia mediante una arquitectura híbrida cliente-nube, con el fin de mitigar la asimetría en la comunicación expresiva y fomentar la autonomía de las personas sordas al interactuar con interlocutores oyentes no señantes.

2.2. Objetivos específicos

- Implementar un módulo de visión computacional y clasificación gestual local basado en la extracción de puntos de referencia corporales (esqueleto y manos) para reconocer un vocabulario de 100 señas de LSA.
- Diseñar e integrar un módulo de traducción semántica bidireccional desplegable como servicio:
 1. Canal directo (LSA $\rightarrow$ Español): transformar secuencias de glosas aisladas en oraciones gramaticalmente fluidas en español rioplatense, considerando contexto conversacional.
 2. Canal inverso (Español $\rightarrow$ LSA): transformar oraciones en español oral a secuencias de glosas estructuradas.
- Implementar un mecanismo de visualización gestual inversa mediante un avatar tridimensional capaz de animar las señas resultantes y deletrear dactilológicamente términos ausentes en el vocabulario.
- Formular y documentar el diseño de una arquitectura escalable en la nube para el aprovisionamiento concurrente de los servicios semánticos, mitigando cuellos de botella en el hardware cliente.
- Diseñar mecanismos heurísticos de segmentación continua (delimitación de inicio y fin de señas, vaciado de búfer de enunciados y compuertas de filtrado de repeticiones).
- Garantizar la privacidad de los usuarios mediante la ejecución de los algoritmos de visión y clasificación de forma local en el dispositivo del cliente, transmitiendo a la nube únicamente cargas textuales desidentificadas.

- Desarrollar la arquitectura de software de una extensión de navegador para plataformas de videoconferencia (Google Meet) articulada con un motor de ejecución en segundo plano (ILSA).

Capítulo 3

Marco teórico, estado del arte y metodología

3.1. Marco teórico

3.1.1. Arquitectura híbrida de partición Cliente-Nube (Edge-Cloud)

Para equilibrar los requerimientos de privacidad, latencia y viabilidad en hardware de consumo masivo, se adoptó una arquitectura de partición cliente-nube (*edge-cloud partitioning*) (Pressman & Maxim, 2014).

En el cliente local (*edge*) se ejecutan la captura de video, la extracción de puntos clave corporales y el modelo de clasificación gestual. Este componente presenta un costo computacional acotado y garantiza que el video crudo nunca abandone el equipo del usuario, resguardando su privacidad e identidad visual.

Por el contrario, los modelos de lenguaje pequeños (SLM) de 1B a 3B parámetros imponen demandas de procesamiento autorregresivo y memoria RAM incompatibles con computadoras estándar que ejecutan simultáneamente una videollamada. Estos módulos se desacoplan y encapsulan como servicios de inferencia en la nube que reciben exclusivamente cargas de texto livianas (vectores de glosas o texto transcripto) y devuelven respuestas contextualizadas con tiempos de transmisión despreciables frente a la latencia de red.

3.1.2. Estructura lingüística de la LSA y traducción Topic-Comment

La LSA es una lengua natural con estatus propio que no comparte la gramática del español (Massone & Martínez, 2012). A diferencia de este último, que sigue preferentemente la estructura Sujeto-Verbo-Objeto (SVO), la LSA carece de artículos, preposiciones gramaticales abstractas y verbos copulativos (“ser” o “estar”) (Massone, 1993; Massone & Martínez, 2012). Morfosintácticamente, la LSA organiza sus proposiciones bajo el principio pragmático de *Topic-Comment* (Tópico-Comentario) (Massone & Martínez, 2012).

Bajo este paradigma, el enunciado sitúa en primer lugar el marco temporal y espacial de referencia (el tópico), seguido por el sujeto, el objeto y finalmente la acción o estado (el comentario):

$$\begin{aligned} \text{Oración LSA} &\rightarrow \text{Tiempo} \rightarrow \text{Lugar} \rightarrow \text{Sujeto} \\ &\rightarrow \text{Objeto/Sustantivo} \rightarrow \text{Verbo} \rightarrow \text{Negación/Pregunta} \end{aligned} \quad (3.1)$$

Por ende, la equivalencia entre ambas lenguas no admite un mapeo léxico término a término. Es imprescindible una etapa intermedia de interpretación semántica que traduzca tanto la secuencia temporal de glosas a español como el texto en español a glosas formalmente ordenadas antes de su animación.

3.1.3. Visión computacional y reconocimiento basado en esqueletos

El procesamiento de video crudo píxel a píxel impone demandas computacionales severas y una alta vulnerabilidad ante variaciones de iluminación, fondos y vestimenta. Siguiendo las directrices del estado del arte, se adopta un esquema basado en coordenadas de articulaciones corporales (*landmarks*) mediante MediaPipe Holistic (Google, 2020).

El sistema extrae 33 puntos de referencia tridimensionales correspondientes a la pose del torso y 21 puntos por cada mano, conformando un vector instantáneo de 225 descriptores numéricos ($[33 + 21 + 21] \times 3$) (Google, 2020). Esta abstracción geométrica independiza el flujo de procesamiento de la apariencia visual directa y reduce la dimensionalidad en varios órdenes de magnitud, facilitando el procesamiento en tiempo pseudo-real en CPU local.

3.1.4. Arquitectura neuronal temporal: TinySkeletonClassifier

El reconocimiento gestual sobre secuencias temporales fijas ($T = 16$ fotogramas) se modela mediante una arquitectura híbrida compacta (413.148 parámetros, $\approx 4,1$ MB de peso) basada en convoluciones y auto-atención (Vaswani et al., 2017; Wu et al., 2020), compuesta por:

1. **Convolución Temporal 1D (Conv1D):** dos capas convolucionales que extraen correlaciones locales inmediatas a lo largo del eje temporal (transiciones rápidas de dedos y muñeca) (Wu et al., 2020).
2. **Codificación Posicional Sinusoidal:** introduce información de orden temporal en la secuencia antes de alimentar al transformador (Vaswani et al., 2017).
3. **Transformer Encoder:** capas de auto-atención multi-cabeza que modelan dependencias de largo alcance entre los instantes iniciales, medios y finales del gesto (Vaswani et al., 2017).
4. **Attention Pooling:** un mecanismo de atención ponderada que calcula un promedio pesado de los estados temporales, priorizando los instantes de pose estática o máxima expresividad por sobre los fotogramas transicionales de aproximación de las manos.

3.1.5. Modelos de lenguaje pequeños (SLM) y adaptación LoRA

Para la traducción semántica bidireccional se emplean Modelos de Lenguaje Pequeños (*Small Language Models*, SLMs) de las familias Qwen2.5 (Alibaba Cloud, 2024), SmoLLM2 (Allal et al., 2024) y Llama 3.2 (Dubey et al., 2024). Estos modelos se adaptan mediante la técnica LoRA (*Low-Rank Adaptation*) (Hu et al., 2022), congelando los pesos base del modelo e insertando matrices de descomposición de bajo rango en las proyecciones de atención y en las capas de entrada y salida (`embed_tokens` y `lm_head`) (Hu et al., 2022; Hugging Face, 2024). Para asegurar su ejecución eficiente en CPU o aceleradores en

la nube, los modelos resultantes se cuantizan a 4 bits bajo el formato GGUF mediante `llama.cpp` (Frantar et al., 2022; Gerganov, 2023).

3.1.6. Animación y representación visual mediante Avatar 3D

El canal inverso requiere transformar las glosas sintácticas en representaciones visuales comprensibles para el usuario sordo. Un avatar tridimensional ejecuta las animaciones asociadas a cada glosa del vocabulario base mediante interpolación de huesos (*skeletal animation*) o blendshapes. Ante términos inexistentes en el diccionario léxico (como nombres propios o tecnicismos), el módulo recurre al subsistema de deletreo manual dactilológico, animando letra por letra la palabra involucrada.

3.2. Estado del arte

3.2.1. Desarrollos en Argentina y la región

En el ámbito nacional, las iniciativas tecnológicas orientadas a la comunidad sorda han estado enfocadas principalmente en la enseñanza y la consulta de vocabulario. El Diccionario de Señas del INJU opera como un índice léxico en línea sin capacidades de reconocimiento dinámico (INJU, 2024). La Confederación Argentina de Sordos (CAS) promueve herramientas pedagógicas bilingües de carácter estático (Confederación Argentina de Sordos, 2024). En cuanto a la animación digital, la aplicación móvil LSApp (Barán, 2018) introdujo un personaje tridimensional capaz de reproducir entradas del diccionario y deletreo manual, funcionando como material de consulta unilateral sin integración a videoconferencias.

En el plano asistencial comercial, la plataforma Blessit (Blessit, 2024) y servicios públicos como la Línea 144 (Ministerio de Justicia de la Nación, 2024) ofrecen teleinterpretación mediante videollamada con operadores humanos. Aunque efectivos, estos esquemas están supeditados a la disponibilidad de personal, generan demoras de conexión y comprometen la intimidad de los interlocutores en intercambios de índole privada.

Académicamente, el corpus LSA64 (Ronchetti et al., 2016) constituye el principal antecedente en reconocimiento automático local; no obstante, su captura controlada mediante guantes de colores restringe su aplicación directa sobre cámaras web comerciales sin aditamentos físicos, limitación observada en desarrollos posteriores con redes recurrentes (Mindlin, 2021).

3.2.2. Sistemas internacionales de reconocimiento y síntesis

A nivel internacional, el referente en reconocimiento unidireccional es SignAll Technologies (Coldewey, 2018; SignAll Technologies, 2026), enfocado en la traducción de ASL hacia inglés escrito mediante visión computacional y modelos lingüísticos. Sin embargo, su enfoque prioriza que el oyente comprenda a la persona sorda, omitiendo la devolución gestual accesible. Por su parte, corpus a gran escala como WLASL (Li et al., 2020) han impulsado el reconocimiento léxico en video de ASL, aunque sin foco en la traducción oracional bidireccional ni en la LSA.

En la dirección inversa (texto a señas), la empresa Hand Talk (Hand Talk, 2025; Sorenson Communications, 2025) lidera la región con avatares 3D (Hugo y Maya) que traducen texto escrito a Libras y ASL. De igual forma, el avatar Iris/Juan desarrollado por la fundación

HETAH (HETAH, 2024) en Colombia realiza síntesis gestual a partir de oraciones en español. No obstante, ninguna de estas plataformas comerciales unifica reconocimiento continuo de LSA, traducción semántica contextual y síntesis gestual interactiva dentro del flujo nativo de una videollamada bajo un esquema híbrido y escalable.

3.3. Metodología de desarrollo

El proyecto se rigió bajo un modelo de ciclo de vida iterativo e incremental (Pressman & Maxim, 2014), estructurado en cinco fases principales:

1. **Curación de datos y formalización léxica:** delimitación de 100 señas de LSA junto a Miriam Rolls; diseño del set de grabación; captura de clips en video 1080p (16:9); y generación del dataset de coordenadas vectoriales normalizadas.
2. **Entrenamiento y diagnóstico del clasificador gestual:** experimentación de arquitecturas temporales (16 vs. 32 fotogramas); optimización de hiperparámetros mediante marcos automatizados (Akiba et al., 2019); desarrollo de instrumentación de diagnóstico de confusiones mediante análisis AUC de Mann-Whitney; y corrección de relaciones de aspecto de cámara.
3. **Desarrollo de los módulos semánticos bidireccionales y diseño de despliegue:** ingeniería de prompts; fine-tuning con LoRA sobre modelos Qwen2.5 (Alibaba Cloud, 2024), SmolLM2 (Allal et al., 2024) y Llama 3.2 (Dubey et al., 2024) (traducción directa e inversa *Topic-Comment* (Massone & Martínez, 2012)); cuantización a binarios GGUF (Gerganov, 2023); evaluación de fidelidad sintáctica (BLEU, ROUGE-L, METEOR); y diseño conceptual de la arquitectura elástica en la nube.
4. **Integración y renderizado gestual:** desarrollo del controlador del avatar 3D para animación léxica y deletreo manual; acoplamiento del pipeline en el motor local ILSA mediante patrones de diseño desacoplados (Gamma et al., 1994); comunicación con el servidor semántico desacoplado; e interceptación de video en Google Meet mediante Canvas sintético y extensión Manifest V3 (Google Chrome Developers, 2024; W3C, 2024).
5. **Fase de pruebas y evaluación de campo:** pruebas integrales de latencia, usabilidad y robustez frente a escenarios reales de videoconferencia.

3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial

En cumplimiento con las directivas de integridad académica y transparencia técnica solicitadas por la comisión evaluadora para Trabajos Finales de Grado de la Facultad de Ingeniería (UNMDP) (Universidad Nacional de Mar del Plata, Facultad de Ingeniería, 2024), los autores detallan el uso de herramientas de inteligencia artificial generativa como soporte durante el desarrollo del proyecto.

Herramientas empleadas Durante las fases de diseño, implementación y redacción se utilizaron exclusivamente dos herramientas:

- **Cursor:** Entorno de desarrollo integrado (IDE) asistido por inteligencia artificial, empleado para la navegación contextual del repositorio, autocompletado sintáctico y asistencia en la edición de scripts en Python y JavaScript.
- **Gemini (Google):** Modelo de lenguaje utilizado como soporte interactivo de consulta técnica, facilitador en la asimilación de conceptos teóricos y asistencia estilística en la redacción del informe.

Alcance y propósito de su utilización La intervención de dichas herramientas se limitó a:

- **Aprendizaje guiado y asimilación conceptual:** Utilización de Gemini como interlocutor de estudio y consulta técnica para comprender en profundidad conceptos que excedían la currícula tradicional de grado, tales como la mecánica de los mecanismos de *attention pooling*, las complejidades de la cuantización GGUF Q4_K_M con `llama.cpp` (Gerganov, 2023), los fundamentos de adaptación de bajo rango (LoRA) en capas densas de modelos autorregresivos (Hu et al., 2022), y los esquemas de comunicación asíncrona entre contextos aislados (*offscreen* y *sandbox*) exigidos por Chrome Manifest V3 (Google Chrome Developers, 2024).
- **Asistencia en el desarrollo de software:** Sugerencias de refactorización de código, resolución de incompatibilidades de sintaxis en el entorno Chrome Extensions Manifest V3 (aislamiento de páginas *offscreen* y *sandbox*) (Google Chrome Developers, 2024) y resolución de dependencias concurrentes en Python.
- **Consultas sobre APIs y formatos:** Apoyo en la parametrización de llamadas a MediaPipe Holistic (Google, 2020) y en la sintaxis de cuantización a formato GGUF mediante herramientas de línea de comandos (Gerganov, 2023).
- **Edición y formato del documento:** Corrección ortotipográfica y resolución de errores en la redacción del informe.

Límites y responsabilidad de autoría Se deja constancia de que ninguna de las herramientas generativas reemplazó la labor intelectual, analítica ni experimental de los autores. La concepción de la arquitectura híbrida, la recolección y curación del corpus de video de 100 señas validado por Miriam Rolls, el diseño e iteración de las redes neuronales (TinySkeletonClassifier), la ingeniería de prompts y fine-tuning de los modelos de lenguaje, el diagnóstico de captura (aspect ratio) y el análisis de resultados fueron ideados, ejecutados y validados íntegramente por los estudiantes. Los autores asumen la total responsabilidad técnica y académica por el contenido del presente informe y del código fuente asociado (Verón & Nigro, 2026).

3.4. Herramientas y tecnologías empleadas

Cuadro 3.1: Conjunto de tecnologías del sistema integrado

Categoría	Herramienta	Rol en el sistema
Entorno y Lenguajes	Python 3.10, JavaScript	Lógica de backend, motor local, servidor semántico y extensión web.
Visión Computacional	MediaPipe Holistic (Google, 2020), OpenCV	Extracción de coordenadas articulares corporales y manipulación de video.
Aprendizaje Profundo	PyTorch 2.6	Inferencia local del clasificador gestual TinySkeletonClassifier.
Modelos de Lenguaje	Transformers (Hugging Face, 2024), PEFT, Unsloth	Fine-tuning eficiente con adaptadores LoRA (Hu et al., 2022) para traducción directa e inversa.
Inferencia de Lenguaje	llama.cpp (Gerganov, 2023), GGUF	Servidor de inferencia semántica optimizado en CPU/GPU remota.
Comunicación y Backend	FastAPI (Ramírez, 2018), Uvicorn	Servidor de interfaz local (ILSA) y microservicio semántico desacoplado.
Túnel de Desarrollo	ngrok / Cloudflare Tunnel	Exposición de la instancia semántica remota para validación del prototipo.
Integración Web	Chrome Extensions (MV3) (Google Chrome Developers, 2024)	Intercepción de media mediante Canvas <code>captureStream()</code> en Google Meet (W3C, 2024).
Síntesis de Salida	pyttsx3, Avatar 3D	Emisión de voz local hacia el oyente y animación visual gestual hacia el señante.

Capítulo 4

Análisis de requerimientos

4.1. Requerimientos funcionales

- **RF-01 — Captura y extracción local de landmarks:** Capturar el flujo de video del usuario señante en el cliente y extraer las coordenadas articulares tridimensionales normalizadas de torso y manos en tiempo real mediante visión computacional (Google, 2020).
- **RF-02 — Clasificación gestual temporal local:** Clasificar localmente secuencias fijas de coordenadas en una de las 100 clases de LSA soportadas y reportar la glosa asignada y su confianza empleando arquitecturas de aprendizaje profundo (Vaswani et al., 2017; Wu et al., 2020).
- **RF-03 — Segmentación heurística de señas:** Identificar de forma automática el inicio y fin de un gesto aislado en función de la actividad de las manos, aplicando políticas de anti-rebote mediante compuertas de repetición (**RepeatGate**) desacopladas (Gamma et al., 1994).
- **RF-04 — Traducción semántica directa remota (LSA $\rightarrow$ Español):** Transmitir las glosas detectadas hacia el servicio semántico en la nube y recibir una oración gramaticalmente cohesiva en español rioplatense generada por un modelo de lenguaje adaptado (Alibaba Cloud, 2024; Hu et al., 2022).
- **RF-05 — Emisión hacia el oyente:** Presentar la traducción en texto superpuesto en pantalla y/o sintetizarla como audio oral para el interlocutor oyente de forma concurrente (Pressman & Maxim, 2014).
- **RF-06 — Captura del turno del interlocutor oyente:** Capturar la voz o texto del oyente en la videollamada durante su turno de intervención para viabilizar el canal de retorno bidireccional.
- **RF-07 — Traducción semántica inversa remota (Español $\rightarrow$ LSA):** Transmitir el texto en español emitido por el oyente hacia el servicio semántico en la nube y obtener una secuencia ordenada de glosas bajo la estructura gramatical *Topic-Comment* canónica de la LSA (Dubey et al., 2024; Massone & Martínez, 2012).
- **RF-08 — Representación visual mediante Avatar LSA:** Recibir las glosas generadas por el canal inverso y renderizar localmente en pantalla la animación

tridimensional de la seña correspondiente, ejecutando deletreo manual dactilológico en términos fuera del léxico (HETAH, 2024; Massone, 1993).

- **RF-09 — Integración con Google Meet:** Inyectar el video procesado y los canales de comunicación dentro de la pestaña de videollamada mediante la derivación de flujo a un lienzo sintético sin generar conflictos de bloqueo en la cámara física (Google Chrome Developers, 2024; W3C, 2024).
- **RF-10 — Memoria conversacional de turnos:** Preservar un historial deslizable con los últimos turnos de diálogo entre ambos participantes para resolver ambigüedades deícticas y pronombres (Gamma et al., 1994).

4.2. Requerimientos no funcionales

- **RNF-01 — Latencia y tiempo de respuesta:** La delimitación de cierre de enunciado debe operar tras un umbral de 4 segundos sin actividad gestual. La inferencia semántica remota y retorno al cliente no debe superar los 1,5 segundos bajo conexiones de banda ancha estándar, apoyándose en la optimización de pesos cuantizados (Frantar et al., 2022; Gerganov, 2023).
- **RNF-02 — Privacidad de imagen y video:** Ningún flujo de video crudo o imagen de la cámara web debe ser transmitido hacia servidores externos o servicios en la nube; el intercambio de red con el backend se restringe exclusivamente a cargas textuales livianas (glosas o texto transcripto), siguiendo principios de diseño seguro (Pressman & Maxim, 2014).
- **RNF-03 — Robustez en reconocimiento en vivo:** El clasificador gestual local debe mantener un rendimiento superior al 75 % top-1 y al 90 % top-3 en condiciones de iluminación doméstica estándar mediante la corrección de distorsiones de aspecto (Ronchetti et al., 2016).
- **RNF-04 — Desacoplamiento y escalabilidad del servicio semántico:** La arquitectura del backend semántico debe estar concebida para operar de forma desacoplada y elástica (Pressman & Maxim, 2014), permitiendo su despliegue en infraestructuras con auto-escalado horizontal a través de interfaces REST tipadas (Ramírez, 2018).
- **RNF-05 — Usabilidad y no invasividad:** La interfaz sobre la videollamada debe ofrecer indicadores visuales de estado (grabando seña, procesando texto, reproduciendo avatar) que no ocluyan la visibilidad del interlocutor, mitigando la brecha comunicacional receptiva y expresiva (República Argentina, 2020).

4.3. Restricciones

- **RC-01 — Alcance léxico:** El vocabulario reconocido y estructurado está acotado a 100 señas validadas por profesionales de LSA, enmarcadas en las pautas pedagógicas y lingüísticas vigentes (Confederación Argentina de Sordos, 2024; INJU, 2024).

- **RC-02 — Despliegue en la nube del prototipo:** Debido a restricciones presupuestarias para el sostenimiento de infraestructura elástica de CPU en proveedores comerciales de nube, la validación del backend semántico remoto se efectúa mediante una instancia dedicada expuesta a través de un túnel seguro (ngrok/Cloudflare), dejándose especificado el diseño arquitectónico de producción escalable (Verón & Nigro, 2026).
- **RC-03 — Compatibilidad de plataforma cliente:** El empaquetado del motor local (ILSA) y los bindings de inferencia están diseñados para el sistema operativo Windows y navegadores basados en Chromium bajo las especificaciones de seguridad vigentes (Google Chrome Developers, 2024).
- **RC-04 — Dependencia de visión:** La robustez del clasificador depende de la capacidad de MediaPipe Holistic para identificar al menos una mano en el cuadro de captura (Google, 2020).
- **RC-05 — Recursos del cliente objetivo:** El componente local del prototipo debe ser capaz de operar con fluidez en laptops estándar (procesadores de 4 núcleos y 8 GB de memoria RAM, sin placa gráfica dedicada), cumpliendo con las pautas de factibilidad establecidas para el Trabajo Final (Universidad Nacional de Mar del Plata, Facultad de Ingeniería, 2024).

4.3.1. Especificación del entorno de ejecución cliente

A fin de asegurar la accesibilidad y democratización del sistema, se definió un hardware objetivo representativo del equipamiento informático estándar presente en ámbitos educativos y laborales:

Cuadro 4.1: Requisitos de hardware y software del dispositivo cliente

Componente	Requisitos mínimos (Dispositivo base)	Requisitos recomendados
Procesador (CPU)	Intel Core i3 (10 ^a gen.) / AMD Ryzen 3 (serie 3000) (4 núcleos / 4 hilos, base $\geq 2,0$ GHz, soporte AVX2).	Intel Core i5 (11 ^a gen.+) / AMD Ryzen 5 (serie 4000+) (6 núcleos / 12 hilos).
Memoria RAM	8 GB DDR4 ($\geq 3,0$ GB libres para el entorno de ejecución).	16 GB DDR4 / DDR5.
Gráficos (GPU)	Gráficos integrados (Intel UHD Graphics 620 / AMD Radeon Vega 3) con soporte WebGL.	Intel Iris Xe / AMD Radeon 600M+ o GPU dedicada básica.
Almacenamiento	Disco de estado sólido (SSD) con 2 GB de espacio libre.	SSD NVMe con ≥ 5 GB libres.
Cámara web	Resolución 720p (1280×720) a 24/30 FPS.	Resolución 1080p a 30 FPS.
Conectividad	Conexión de banda ancha (≥ 10 Mbps bajada / ≥ 3 Mbps subida, latencia < 80 ms).	Conexión por fibra óptica (≥ 50 Mbps).
Entorno de software	Windows 10/11 (64 bits), Google Chrome 115+ (soporte Manifest V3).	Windows 11 (64 bits), Google Chrome última versión estable.

Criterio técnico de selección del procesador base La especificación de la CPU base no fue adoptada de manera arbitraria, sino deducida a partir de la descomposición de hilos requerida en ejecución simultánea:

1. **Carga WebRTC en navegador:** Google Meet demanda aproximadamente entre 1 y 1,5 núcleos para la decodificación y codificación de video en tiempo real.
2. **Extracción (MediaPipe Holistic):** La ejecución sobre WebAssembly en el contexto aislado del navegador insinúa la capacidad de 1 núcleo dedicado para mantener una cadencia de 24 a 30 FPS.
3. **Inferencia gestual e interprocesos (Motor ILSA):** El servidor local FastAPI y el modelo neuronal en CPU (*TinySkeletonClassifier*) requieren disponibilidad de cómputo vectorial (instrucciones AVX2/FMA) para garantizar latencias de clasificación inferiores a los 2 ms sin contienda de recursos.

Esta descomposición fundamenta la exigencia de una arquitectura mínima de 4 núcleos físicos y frecuencia base sostenida de 2,0 GHz, viable exclusivamente gracias al desacoplamiento del modelo de lenguaje autorregresivo hacia el backend en la nube.

4.4. Casos de uso

4.4.1. Casos de uso esenciales en formato breve

- **CU-01 — Conversar con otra persona (Modo Sordo):** El usuario sordo seña frente a la cámara web durante la videollamada; el sistema captura y clasifica localmente las señas, solicita la traducción remota y proyecta el mensaje como texto o voz para el oyente (Pressman & Maxim, 2014).
- **CU-02 — Configurar mano dominante:** El usuario señante selecciona si su mano hábil es diestra o zurda para que el sistema normalice adecuadamente los vectores articulares mediante operaciones de espejado (Google, 2020).
- **CU-03 — Configurar canal de salida del oyente:** El usuario oyente selecciona si prefiere recibir los mensajes del señante en formato de subtítulos, voz sintetizada en off o ambas alternativas simultáneamente.
- **CU-04 — Traducir español a LSA mediante Avatar (Modo Oyente):** El usuario oyente habla o escribe durante la llamada; el sistema solicita la estructuración remota en glosas LSA y el avatar reproduce localmente las señas o las deletrea visualmente (HETAH, 2024; Massone, 1993).

4.4.2. Casos de uso expandidos

Identificador	CU-01: Conversar con otra persona (Modo Sordo)		
Actor principal	Usuario señante (persona sorda)		
Precondiciones	El motor ILSA está activo (Ramírez, 2018), conectado al servicio semántico remoto y el usuario se encuentra en la videollamada con cámara encendida.		
Postcondiciones	El mensaje en LSA es traducido a español e interpretado como texto o voz para el oyente.		
Escenario principal de éxito		Usuario señante	Sistema
	1	Seña frente a la cámara su mensaje de forma continua.	
	2		Detecta las manos, segmenta cada seña, extrae coordenadas (Google, 2020) y clasifica localmente las glosas (Wu et al., 2020).
	3	Realiza una pausa de al menos 4 segundos al finalizar su turno.	
	4		Cierra el enunciado y transmite las glosas al servicio semántico en la nube (Hu et al., 2022).
	5		Recibe la oración traducida, proyecta el subtítulo y reproduce el audio en el cliente.
Extensiones	2a. Glosa con baja confianza o descartada por rebote: se omite el registro y se reinicia la espera (Gamma et al., 1994). 4a. Falla de conectividad con el servicio semántico: se emite el texto concatenado de las glosas directas como contingencia.		

Identificador	CU-04: Traducir español a LSA mediante Avatar (Modo Oyente)		
Actor principal	Usuario no señante (persona oyente)		
Precondiciones	La videollamada está establecida, el modo oyente se encuentra activo y hay conexión con el servicio semántico remoto.		
Postcondiciones	El mensaje hablado es visualizado en LSA por el usuario sordo mediante el avatar 3D (HETAH, 2024).		
Escenario principal de éxito		Usuario no señante	Sistema
	1	Habla a través del micrófono o ingresa texto en la interfaz.	
	2		Transcribe el flujo de audio a texto en español.
	3		Envía el texto al servicio semántico inverso en la nube (Dubey et al., 2024).
	4		Recibe la secuencia de glosas estructurada bajo el formato <i>Topic-Comment</i> (Massone & Martínez, 2012).
	5		El Avatar 3D reproduce las señas en pantalla y deletrea términos fuera del vocabulario (Massone, 1993).
Extensiones	5a. Término no reconocido en el léxico base: el avatar activa la animación de deletreo dactilológico letra a letra (Massone, 1993).		

Capítulo 5

Diseño e Implementación de la Arquitectura del Sistema

5.1. Arquitectura global y partición Cliente-Nube

5.1.1. Justificación de la partición y modelo de privacidad

La delimitación de la frontera entre el procesamiento local y los servicios remotos constituyó una de las decisiones arquitectónicas centrales del proyecto. Frente al requerimiento de aligerar la carga de cómputo en el equipo del usuario sin comprometer la latencia ni la privacidad, se evaluaron dos alternativas de diseño:

- **Alternativa A (Clasificación remota):** El cliente extrae las coordenadas articulares con MediaPipe y transmite los *landmarks* al servidor, donde se ejecutan tanto la clasificación gestual como la interpretación semántica.
- **Alternativa B (Clasificación local):** El cliente ejecuta localmente la extracción de puntos y la clasificación gestual (*TinySkeletonClassifier*), transmitiendo al servidor únicamente las glosas textuales reconocidas para su traducción con el modelo de lenguaje.

A continuación se detallan las razones técnicas que fundamentaron la adopción de la Alternativa B.

Asimetría de cómputo y viabilidad en CPU cliente. Al evaluar cómo respondía cada parte en la práctica, comprobamos que el esfuerzo de cómputo estaba totalmente desbalanceado. El modelo *TinySkeletonClassifier* posee 413.148 parámetros y un peso en disco de aproximadamente 4,1 MB. Su inferencia sobre una secuencia de 16 fotogramas normalizados insume menos de 2 ms en una CPU convencional de gama media, lo que significa un consumo mínimo que convive perfectamente con la videollamada en segundo plano.

Por el contrario, ejecutar modelos de lenguaje de 1B a 3B parámetros (incluso cuantizados a 4 bits bajo formato GGUF) requiere entre 0,8 GB y 2,2 GB de memoria y miles de operaciones matriciales por token. La coexistencia simultánea de Google Meet, la extracción de puntos y los SLM en una computadora portátil con 8 GB de RAM provocaba saturación de memoria y demoras de hasta 5 segundos por turno. En consecuencia, la única sobrecarga crítica radicaba en el modelo de lenguaje, justificando mantener el clasificador en el borde.

Optimización del tráfico de red y latencia de realimentación. La transmisión continua de secuencias de coordenadas articulares (Alternativa A) exigiría enviar por cada seña un tensor de (16, 225) números de punto flotante. Serializado en formato JSON, este paquete alcanza entre 45 KB y 80 KB por intento de gesto, obligando a sostener una conexión persistente de alta frecuencia hacia el servidor.

Bajo la arquitectura adoptada (Alternativa B), el clasificador ligero y la compuerta **RepeatGate** resuelven el filtrado directamente en memoria local: las predicciones de baja confianza o descartadas por rebote se eliminan en el cliente, de modo que hacia la nube únicamente se despachan enunciados cerrados compuestos por las glosas consolidadas que superaron los umbrales de validación. Esto reduce la carga útil enviada a través de internet, disminuyendo el tráfico de red y evitando demasiadas peticiones al backend semántico. Asimismo, la clasificación local permite brindar retroalimentación visual inmediata al usuario señante en la interfaz superpuesta (HUD): el sistema confirma la detección de la glosa en tiempo real (latencia < 2 ms), sin depender de la fluctuación del retardo de ida y vuelta de red (RTT) que introduciría una clasificación remota.

Filtrado local de ruido y compuertas anti-rebote. La segmentación gestual en entornos cotidianos produce falsos positivos y repeticiones involuntarias. Al clasificar en el cliente, las políticas heurísticas de descarte (confianza mínima, umbrales de manos activas y la compuerta **RepeatGate**) se resuelven en memoria local. Esto evita saturar el servidor remoto, enviando hacia la nube únicamente enunciados cerrados y convalidados tras el período de pausa (4 segundos).

Modelo de privacidad por diseño (*Privacy by Design*). Desde una perspectiva de seguridad y confidencialidad, las coordenadas articulares tridimensionales constituyen representaciones biométricas indirectas capaces de reflejar la fisonomía y la dinámica corporal del usuario. Al retener el flujo de video y la totalidad de los vectores cinemáticos en el cliente, se garantiza el principio de minimización de datos: la infraestructura en la nube opera de forma ciega a la fisonomía del individuo, procesando exclusivamente texto desidentificado.

5.1.2. Flujo integral de datos entre componentes.

La dinámica operativa del sistema involucra procesos asíncronos y desacoplados que conectan el navegador web, el entorno de ejecución local en el cliente (*edge*) y los microservicios semánticos remotos alojados en la infraestructura en la nube (*cloud*). La Figura 5.1 ilustra la topología física y funcional de la solución, delimitando las fronteras de red y las responsabilidades asignadas a cada entorno.

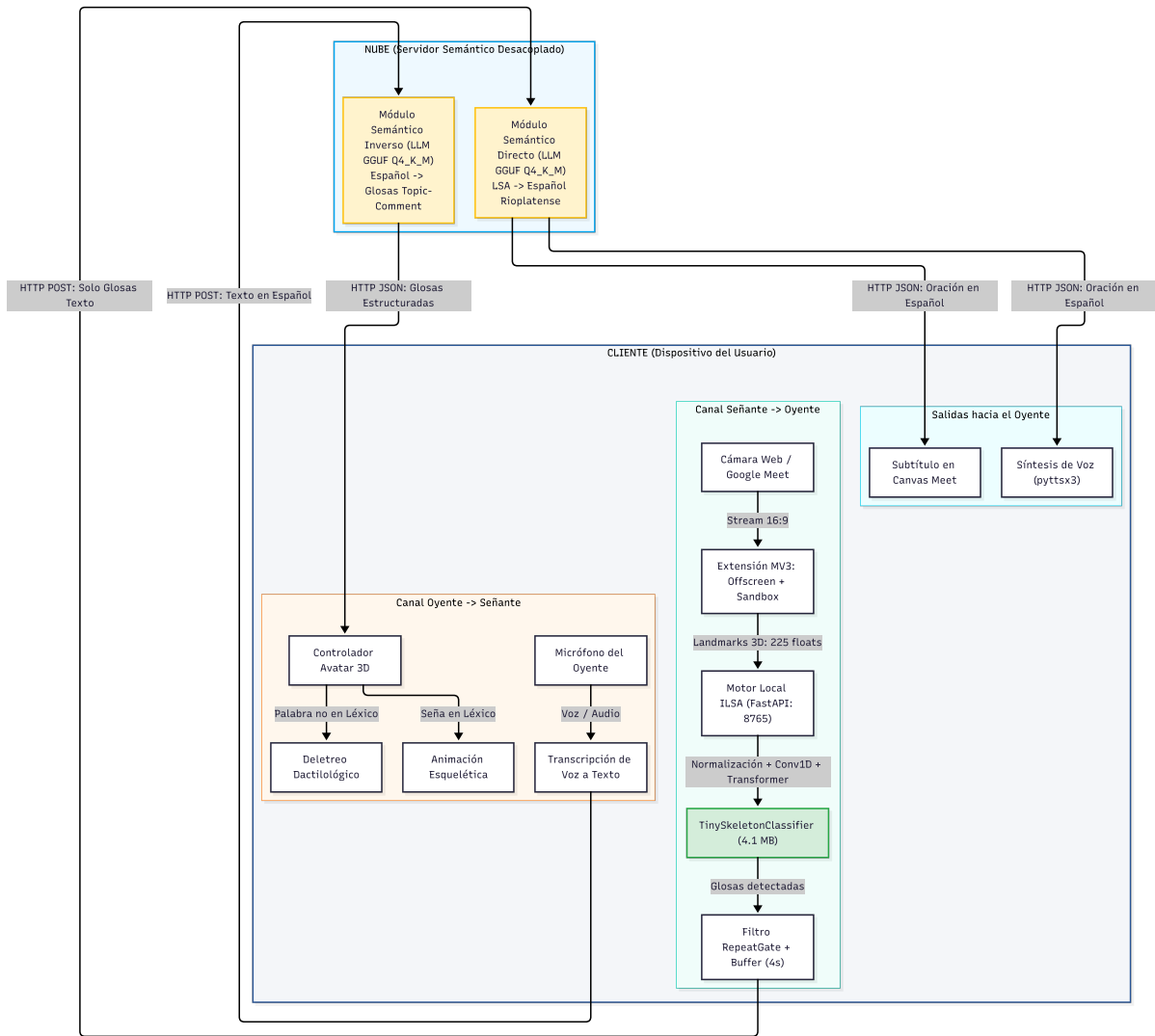


Figura 5.1: Arquitectura global de integración: partición entre el dispositivo cliente (captura, extracción de *landmarks*, clasificación y avatar 3D) y el servidor semántico desacoplado en la nube.

Para modelar la interacción temporal y la secuencia de mensajes se analizan a continuación los dos canales comunicacionales de manera independiente.

Dinámica temporal del Canal Señante → Oyente (Modo Sordo)

En la Figura 5.2 se modela el ciclo de vida completo de un enunciado en LSA, desde la articulación gestual frente a la cámara web hasta la emisión de subtítulos y voz sintética hacia el interlocutor oyente.

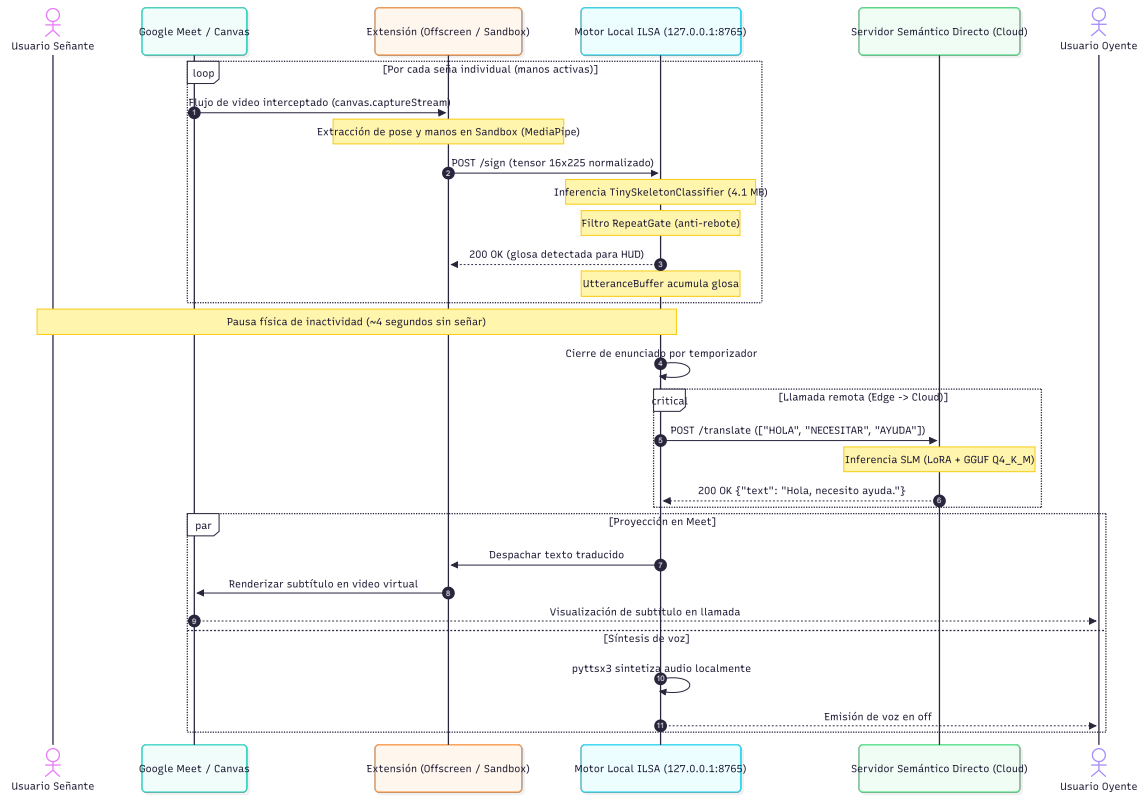


Figura 5.2: Diagrama de secuencia del Canal Señante → Oyente: bucle continuo de clasificación de señas en el borde, delimitación heurística de pausas de 4 segundos e invocación al servidor semántico remoto.

El procesamiento de este canal se rige bajo la siguiente secuencia de eventos técnicos:

1. **Intercepción de video y aislamiento en la extensión:** Al activarse el sistema, la extensión intercepta la llamada a `getUserMedia` en Google Meet mediante la derivación de flujo a un elemento `<canvas>` virtual con el método `captureStream()` (W3C, 2024). Para sortear las restricciones del entorno Chrome Manifest V3 sin degradar el hilo principal de la videollamada (Google Chrome Developers, 2024), los cuadros de video se envían a un documento *offscreen* que delega en un *sandbox* aislado la inferencia de MediaPipe Holistic (Google, 2020), obteniendo los 225 puntos articulares tridimensionales correspondientes a la pose y ambas manos.
2. **Clasificación temporal continua en el motor local (ILSA):** Por cada seña aislada delimitada por la presencia y cinemática de las manos (con umbrales temporales ajustados a $\approx 0,9$ s de inmovilidad o gracia de 0,4s ante pérdidas momentáneas de seguimiento), la extensión serializa la secuencia normalizada de 16 fotogramas y la envía mediante una petición HTTP POST (`/sign`) al servidor local FastAPI que corre en 127.0.0.1:8765.
3. **Inferencia ligera y filtrado heurístico (RepeatGate):** El modelo neuronal *TinySkeletonClassifier* (4,1 MB) ejecuta su inferencia en CPU en menos de 2 ms. La predicción atraviesa la compuerta de validación *RepeatGate* (Gamma et al., 1994), la cual aplica políticas específicas según la categoría léxica de la glosa: admite repeticiones

indefinidas para dígitos numéricos, un máximo de dos apariciones consecutivas para letras individuales del alfabeto, y descarta repeticiones inmediatas en vocabulario general para suprimir rebotes del clasificador. La glosa aceptada se envía de retorno a la extensión para actualizar en tiempo real el HUD de confirmación del señante y se apila en el acumulador `UtteranceBuffer`.

4. **Temporización y cierre de enunciado:** El acumulador monitorea la actividad gestual. Cuando el usuario retira las manos o permanece en reposo por un intervalo continuo de 4 segundos, el temporizador de inactividad declara el cierre del enunciado.
5. **Invocación remota al servidor semántico:** El motor local ILSA transmite exclusivamente la lista de glosas estructuradas (payload en texto de menos de 100 bytes) hacia el servidor semántico en la nube mediante un POST a `/translate`. El modelo de lenguaje pequeño adaptado con LoRA (Hu et al., 2022) y cuantizado a GGUF Q4_K_M (Gerganov, 2023) resuelve la coherencia gramatical en español rioplatense.
6. **Emisión concurrente de resultados:** La oración traducida en ILSA, se bifurca en dos salidas paralelas: se transmite a la extensión para ser dibujada como subtítulo persistente durante 8 segundos sobre el `<canvas>` de video transmitido a Meet (W3C, 2024), y en simultáneo se envía al hilo secundario de `pyttsx3` para su reproducción sintetizada por voz hacia el oyente sin bloquear el flujo visual.

Dinámica temporal del Canal Oyente → Señante (Modo Oyente)

En la Figura 5.3 se describe la secuencia temporal que transforma el mensaje oral del oyente en representaciones visuales comprensibles en LSA para el usuario sordo.

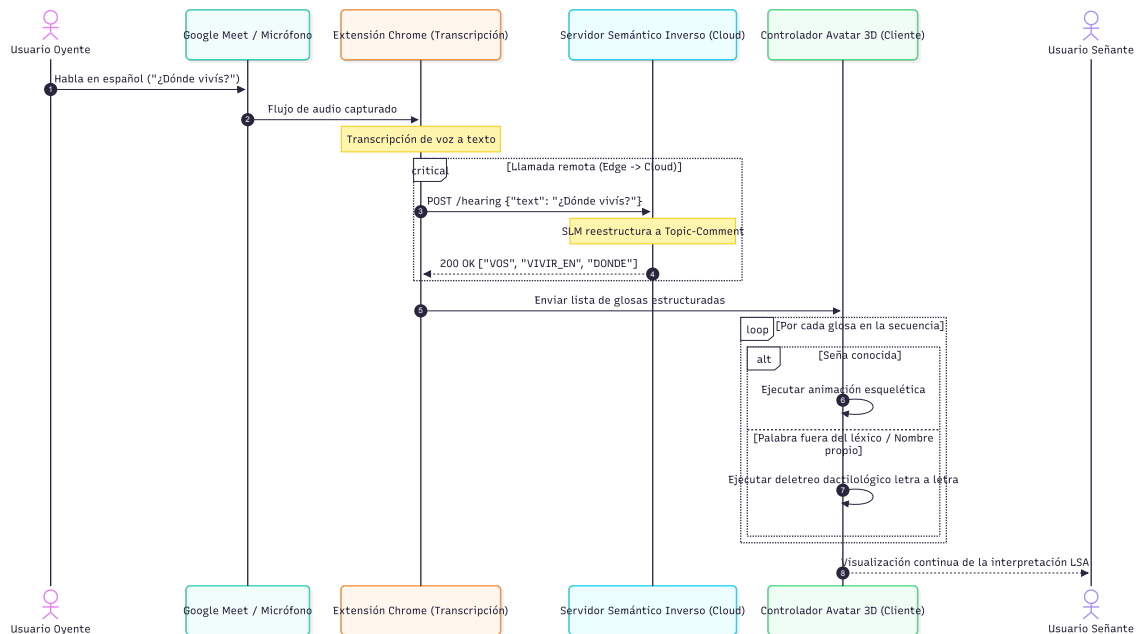


Figura 5.3: Diagrama de secuencia del Canal Oyente → Señante: transcripción del habla, reestructuración gramatical Topic-Comment en la nube y animación o deletreo dactilológico en el cliente.

El canal inverso se desarrolla mediante los siguientes pasos técnicos:

1. **Captura y transcripción del audio:** Durante el turno del interlocutor oyente, el audio capturado en la pestaña de la videollamada es procesado por los mecanismos de reconocimiento de voz de la extensión, obteniendo la transcripción textual de la oración en español.
2. **Estructuración semántica inversa en la nube:** La cadena textual se envía mediante HTTP POST (`/hearing`) al microservicio semántico inverso remoto. El modelo especializado aplica el conjunto de reglas aprendidas para despojar al mensaje de artículos y preposiciones abstractas, reordenando las unidades léxicas bajo la estructura gramatical *Topic-Comment* propia de la LSA (Massone & Martínez, 2012):

$$\text{Español: "¿Dónde vivís?"} \longrightarrow \text{LSA: ["VOS", "VIVIR_EN", "DONDE"]} \quad (5.1)$$

3. **Despacho y ejecución sobre el Avatar 3D local:** La lista ordenada de glosas retorna a la extensión en el cliente, la cual invoca secuencialmente al controlador del avatar tridimensional. Para cada elemento de la secuencia:
 - Si la glosa pertenece al vocabulario conocido, el motor gráfico ejecuta la interpolación esquelética correspondiente al gesto tridimensional.
 - Si el vocablo corresponde a un término no registrado en el léxico base o a un nombre propio, el controlador acciona de manera automática la subrutina de deletreo manual dactilológico, animando letra por letra la palabra involucrada.

El usuario sordo visualiza de este modo la traducción continua en su lengua natural de manera sincronizada sobre la interfaz de la llamada.

5.1.3. Esquema de comunicación y contrato de interfaces (API REST)

La integración de los componentes del sistema se articula a través de dos interfaces de programación de aplicaciones (API) basadas en el estilo arquitectónico REST, implementadas sobre el framework asíncrono FastAPI y montadas mediante servidores ASGI Uvicorn (Ramírez, 2018):

1. **API Local del Motor ILSA (127.0.0.1:8765):** Expuesta en la interfaz de bucle invertido (*loopback*) del cliente, actúa como intermediaria de control entre la extensión de Chrome y los procesos locales de clasificación gestual y síntesis auditiva.
2. **API del Servicio Semántico Remoto (*Cloud*):** Microservicio desacoplado accesible a través de una URL, responsable de resolver la traducción contextual directa e inversa mediante modelos semánticos.

Políticas de seguridad, CORS y enlace de red Por motivos de seguridad y privacidad estricta, el servidor local ILSA se enlaza exclusivamente a la dirección IP 127.0.0.1, denegando peticiones originadas fuera del dispositivo cliente. Para habilitar la comunicación desde la videollamada sin vulnerar las restricciones del navegador, se implementa una política de Intercambio de Recursos de Origen Cruzado (*CORS*) restrictiva que admite únicamente peticiones provenientes del origen de la plataforma (<https://meet.google.com>), orígenes con el esquema de extensión del navegador (`chrome-extension://*`) y `localhost`.

Contratos de la API Local ILSA

La interacción entre la extensión y el motor local se rige por contratos tipados mediante esquemas Pydantic (Ramírez, 2018). La Tabla 5.1 resume los principales endpoints provistos por el motor local.

Cuadro 5.1: Endpoints principales de la API local del motor ILSA (127.0.0.1:8765)

Método	Ruta	Esquema / Parámetros	Descripción y Comportamiento
GET	/health	Ninguno	Monitoreo del estado operativo del clasificador y conectividad con el traductor semántico.
GET	/state	Ninguno	Devuelve las glosas acumuladas en el búfer temporal y el último texto traducido.
POST	/session	SessionStart-Request	Inicia una sesión de señado configurando la lateralidad (<code>left_handed: bool</code>).
POST	/sign	SignPayload	Ingesta una lista de 16 fotogramas con 225 coordenadas articulares normalizadas; ejecuta el clasificador y el filtro RepeatGate.
POST	/activity	Ninguno	Notifica actividad cinemática para resetear el temporizador de cierre de enunciado.
POST	/utterance/end	Ninguno	Cierra forzosamente el enunciado actual, despacha las glosas acumuladas al traductor y vacía el búfer.
POST	/conversation/clear	Ninguno	Reinicia el contexto conversacional de turnos cruzados.

Estructura de la carga de entrada para clasificación (/sign) El endpoint /sign consume un objeto JSON con la secuencia cinemática procesada en el navegador:

```
{
  "frames": [
    [0.012, -0.451, ..., 0.128],    // 225 flotantes del fotograma 0
    ...,
    [-0.005, -0.442, ..., 0.119]  // 225 flotantes del fotograma 15
  ],
  "source_fps": 30.0
}
```

En respuesta, el motor valida el gesto y devuelve un objeto de confirmación que alimenta el visor de estado (HUD) de la extensión:

```
{
  "status": "accepted",
  "gloss": "HOLA",
}
```

```

"confidence": 0.942,
"top3": [
  {"gloss": "HOLA", "prob": 0.942},
  {"gloss": "CHAU", "prob": 0.031},
  {"gloss": "VER", "prob": 0.015}
],
"utterance_closed": false
}

```

Contratos de la API del Servicio Semántico Remoto

El microservicio en la nube centraliza la inferencia del modelo de lenguaje a través de dos rutas, resumidas en la Tabla ??.

Identificador	CU-01: Conversar con otra persona		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	El usuario no señante es capaz de leer/escuchar lo que el usuario señante le dice.		
Escenario Principal de éxito (flujo básico)		Usuario señante	Sistema
	1	Realiza las señas correspondientes a lo que quiere comunicar.	
	2		El Sistema procesa el búfer con las señas, extrae los landmarks, interpreta cada gesto y envía la secuencia al módulo semántico, el cual traduce y muestra por pantalla la oración resultante.
Extensiones	—		
Requisitos especiales	—		

Contrato del Canal Directo (/translate) La petición generada por el motor local hacia el servidor remoto encapsula las glosas detectadas y el historial deslizante:

```

// Solicitud POST /translate
{
  "glosses": ["AHORA_HOY", "YO", "BIEN"],
  "history": [
    {"role": "hearing", "text": "¿Cómo estás hoy?"}
  ]
}

// Respuesta 200 OK
{

```

```

    "spanish": "Hoy estoy bien.",
    "model_used": "llama-3.2-1b-instruct-lsa-q4_k_m",
    "inference_time_ms": 348.3
}

```

Contrato del Canal Inverso (/hearing) En sentido inverso, el texto transcripto de la voz del oyente se remite al modelo inverso para su desarticulación gramatical:

```

// Solicitud POST /hearing
{
    "text": "¿Dónde vivís actualmente?"
}

// Respuesta 200 OK
{
    "glosses": ["VOS", "VIVIR_EN", "DONDE"],
    "inference_time_ms": 337.7
}

```

Este desacoplamiento contractual garantiza una delimitación de responsabilidades limpia: el motor cliente no depende de la arquitectura interna ni de los pesos del modelo de lenguaje que atiende en la API remota, requiriendo únicamente el cumplimiento estricto del esquema de intercambio JSON.

5.2. Subsistema de reconocimiento gestual (Clasificador)

5.2.1. Captura e ingesta de video con MediaPipe Holistic

5.2.2. Normalización geométrica y corrección de distorsión de relación de aspecto

5.2.3. Preprocesamiento: recorte temporal, submuestreo e interpolación

5.2.4. Arquitectura neuronal TinySkeletonClassifier (Conv1D + transformer + attention pooling)

5.2.5. Segmentación continua y políticas anti-rebote (RepeatGate)

5.3. Subsistema de traducción semántica bidireccional

5.3.1. Canal directo (LSA $\rightarrow$ Español): fine-tuning LoRA y loss masking

El canal directo tiene como propósito transformar secuencias de glosas aisladas provenientes del clasificador gestual en oraciones sintáctica y semánticamente correctas en

español rioplatense natural. A diferencia de una correspondencia léxica término a término, este módulo debe inferir artículos, preposiciones, flexiones verbales y terminaciones de género que no se encuentran explícitas en la estructura de entrada de las señas.

Estructuración léxica y reglas de traducción En la fase de refinamiento se suprimieron los ejemplos demostrativos fijos (*few-shots*) en el prompt, optando por un *system prompt* compacto y estricto. Sobre el vocabulario formalizado de 100 clases, las directivas lingüísticas integradas regulan:

- **Marcadores temporales y flexión verbal:** Las glosas temporales iniciales condicionan la conjugación de los verbos subsiguientes. Glosas como PASADO o AYER obligan al uso de pretérito, FUTURO rige el tiempo futuro y la ausencia de marcador temporal se traduce por defecto en presente del indicativo.
- **Resolución de género en parentescos mediante dactilología:** De acuerdo con los estudios de morfología en LSA (Massone, 1993), los sustantivos no suelen tener una terminación fija para masculino o femenino. Por eso, en los términos de parentesco, se adoptó la combinación de la seña base con las letras dactilológicas [A] u [O] para marcar explícitamente el género femenino o masculino:

["HIJO", "A"]	→	“Hija”
["HIJO", "O"]	→	“Hijo”
["HERMANO", "A"]	→	“Hermana”
["ESPOSO", "A"]	→	“Esposa”

El modelo semántico interpreta esta letra añadida y ajusta la terminación de la palabra en español. Ante la ausencia de una marca explícita, genera la forma masculina o neutra por defecto.

- **Fórmulas de interacción, auxilio y cortesía:** La inclusión de glosas como AYUDAR y GRACIAS permite estructurar pedidos de auxilio y fórmulas de interacción esenciales en situaciones de emergencia o diálogo cotidiano (e.g., ["GRACIAS"] → “Gracias”; ["VOS", "YO", "AYUDAR", "PODER"] → “¿Me podés ayudar?”; ["YO", "VOS", "AYUDAR"] → “Te ayudo”).
- **Uso de posesivos:** Las glosas MIO y TUYO se traducen de forma natural en español según el objeto que acompañen (por ejemplo, «mi documento» o «tu casa»).

Mecanismo de Loss Masking (train_on_responses_only). En el entrenamiento estándar de modelos sobre pares de instrucción y respuesta, la función de pérdida por entropía cruzada se computa sobre la totalidad de los tokens de la secuencia. Esto obliga a la red a predecir tanto las instrucciones del sistema como las glosas de entrada.

Para optimizar el uso del gradiente, se implementó la técnica de enmascaramiento de pérdida (*loss masking*) mediante la directiva `train_on_responses_only` (Hugging Face, 2024). En el cálculo de la función objetivo, los tokens correspondientes al prompt del sistema y a la lista de glosas son etiquetados con el identificador `-100`, ignorándolos en la optimización, donde el cómputo del error recae exclusivamente sobre los M tokens de la oración final en español condicionada por el contexto. De este modo, la actualización de pesos se concentra por completo en la fluidez y precisión gramatical del texto generado.

Adaptación LoRA profunda y proyecciones de vocabulario. La adaptación eficiente de bajo rango (*LoRA*) descompone las matrices de pesos preentrenadas $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ en dos matrices entrenables $\mathbf{B} \in \mathbb{R}^{d \times r}$ y $\mathbf{A} \in \mathbb{R}^{r \times k}$ (donde $r \ll \min(d, k)$), actualizando los pesos según $\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA})$ (Hu et al., 2022).

Además de inyectar adaptadores en las matrices de proyección de atención convencionales ($\mathbf{q_proj}$, $\mathbf{k_proj}$, $\mathbf{v_proj}$, $\mathbf{o_proj}$), la arquitectura del traductor incorporó de forma deliberada adaptadores LoRA sobre los extremos del modelo:

- **embed_tokens:** Adapta la capa de embedding de entrada para que las 100 glosas en mayúsculas se proyecten en representaciones semánticas que capturen la estructura de unidades de lengua de señas.
- **lm_head:** Ajusta la capa lineal de desproyección final hacia el vocabulario, facilitando que la red asigne probabilidades a artículos, preposiciones y terminaciones verbales del español rioplatense.

Configuración hiperparamétrica del entrenamiento Para estabilizar el proceso de optimización frente al volumen acotado del dataset se adoptaron los siguientes hiperparámetros de entrenamiento:

- Parámetros de adaptación LoRA: rango de descomposición $r = 16$ y factor de escala $\alpha = 32$ (incrementado para otorgar mayor ponderación relativa a las matrices residuales adaptadas) (Hu et al., 2022).
- Tasa de aprendizaje (*learning rate*): valor base de $\eta = 5 \times 10^{-5}$ con decaimiento coseno suave (*cosine learning rate scheduler*).
- Esquema de épocas y lotes: entrenamiento establecido en 10 épocas completas con un tamaño de lote efectivo de 4 muestras procesadas mediante acumulación de gradientes.

5.3.2. Canal inverso (Español $\rightarrow$ LSA): estructuración gramatical topic-comment

El canal inverso tiene como propósito resolver la interpretación simétrica del mensaje: transformar el texto en español oral o escrito emitido por el interlocutor oyente hacia una secuencia estructurada de glosas sintácticas. Al igual que en el canal directo, esta operación no admite un reemplazo directo palabra por palabra, ya que exige desarmar la estructura de la oración en español y reordenar las palabras según la gramática de la LSA.

Asimetría de dominios y desafío de generalización abierta. Durante las primeras iteraciones se asumió que el modelo inverso podía entrenarse invirtiendo simétricamente el corpus paralelo utilizado para el canal directo. Sin embargo, la experimentación reveló una asimetría metodológica crítica entre ambos extremos comunicacionales:

- **Canal directo (Dominio cerrado):** La entrada está rígidamente acotada por construcción a las 100 clases que el clasificador visual es capaz de reconocer. El modelo solo debe aprender a combinar e infleccionar ese léxico cerrado hacia el español.

- **Canal inverso (Dominio abierto):** La entrada corresponde al habla o texto libre del interlocutor oyente, el cual contiene un léxico virtualmente infinito: sinónimos no contemplados, giros coloquiales, tiempos compuestos y estructuras sintácticas complejas.

Entrenar el modelo con un conjunto limitado a oraciones espejo construidas únicamente con las 100 palabras provocaba un sobreajuste severo: ante vocablos no previstos, nombres propios o frases indirectas del oyente, el modelo colapsaba o generaba alucinaciones intentando forzar glosas conocidas.

Metodología de refinamiento impulsada por fallos (*Failure-Driven Loop*). Para sortear esta limitación sin incurrir en costos inabordables de anotación manual, se implementó un ciclo iterativo de ingeniería y aprendizaje activo:

1. **Entrenamiento inicial:** Ajuste con LoRA sobre el corpus inverso de partida.
2. **Evaluación con casos límite:** Pruebas automáticas sobre oraciones variadas para evaluar la flexibilidad del modelo ante sinónimos y estructuras complejas en español.
3. **Análisis de fallas recurrentes:** Identificación de fallas repetidas, como dejar preposiciones que no se señan, ubicar mal las preguntas al inicio o invertir el orden de la acción.
4. **Corrección del dataset:** Agregado de nuevos pares de oraciones enfocados puntualmente en los patrones que causaban error.
5. **Reentrenamiento y control:** Reentrenamiento del adaptador y comparación cuantitativa de métricas sintácticas (BLEU, ROUGE-L) y latencia.

Estructuración gramatical y reglas del System Prompt. El comportamiento del modelo de lenguaje se rige mediante un *system prompt* estricto que instruye al modelo a responder exclusivamente con glosas léxicas en mayúsculas, separadas por espacios, sin signos de puntuación adicionales ni artículos en español. Las directivas morfosintácticas integradas exigen:

1. **Supresión de elementos no señados:** Eliminación sistemática de artículos determinados e indeterminados (“el”, “la”, “los”, “un”), cópulas abstractas (“ser”, “estar”) y preposiciones gramaticales vacías (“de”, “a”, “con”) (Massone & Martínez, 2012).
2. **Orden canónico Topic-Comment:** Reestructuración pragmática de la oración siguiendo el orden jerárquico de la LSA (Massone & Martínez, 2012):

$$\begin{aligned} \text{Orden LSA} = & \text{TIEMPO} \longrightarrow \text{LUGAR} \longrightarrow \text{SUJETO} \longrightarrow \text{OBJETO/SUST.} \\ & \longrightarrow \text{VERBO} \longrightarrow \text{PREGUNTA / NEGACIÓN} \end{aligned} \quad (5.2)$$

3. **Marcación temporal inicial:** Las referencias temporales explícitas o deducidas de la conjugación se extraen y posicionan obligatoriamente en el arranque del enunciado mediante glosas como PASADO, FUTURO, AYER o AHORA_HOY.

4. **Explicitación pronominal:** En español es frecuente el sujeto tácito; la LSA requiere la presencia de los pronombres de primera y segunda persona (YO, VOS), por lo que el modelo infiere y explicita estos índices cuando están elididos (e.g., “*Tengo un cuchillo*” → [“YO”, “CUCHILLO”, “TENER”]) (Massone, 1993).
5. **Cierre con negación e interrogativos:** Las partículas interrogativas (QUÉ, DÓNDE, QUIÉN, CUÁNDO, CUÁNTOS) y los marcadores de polaridad negativa (NO) se desplazan a
(Massone & Martínez, 2012).

Este procesamiento asegura que la secuencia de glosas resultante refleje fielmente la organización espacial y mental de la lengua de señas antes de ser despachada al controlador del avatar 3D.

5.3.3. Optimización de inferencia: formato GGUF y cuantización a 4 bits

La integración de SLMs en una arquitectura interactiva orientada a videollamadas impone restricciones severas sobre la memoria volátil y la latencia de respuesta por token. Ejecutar modelos de lenguaje en precisión estándar de punto flotante de 16 bits (`float16`) demanda entre 2,5 GB y más de 6 GB de memoria VRAM/RAM dedicada únicamente a los pesos, lo cual resulta prohibitivo para entornos locales compartidos con decodificación de video WebRTC. Para garantizar una inferencia rápida y de bajo consumo, se implementó un pipeline de exportación y compresión post-entrenamiento hacia el formato unificado GGUF (*GPT-Generated Unified Format*) ejecutado mediante el runtime `llama.cpp` (Gerganov, 2023).

Consolidación de adaptadores y exportación. Una vez finalizado el ajuste fino supervisado del modelo con adaptadores LoRA, los pesos residuales de bajo rango no se cargan de manera dinámica en el entorno de producción. En su lugar, se realiza una fusión determinista (*weight merging*) aplicando la rutina `save_pretrained_gguf`:

$$\mathbf{W}_{\text{final}} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA}) \quad (5.3)$$

donde $\mathbf{W}_0$ corresponde a los tensores base preentrenados del SLM (e.g., Llama 3.2 1B (Dubey et al., 2024) o Qwen2.5 (Alibaba Cloud, 2024)) y $\mathbf{BA}$ representa la descomposición adaptada de las proyecciones de atención y capas de vocabulario (`embed_tokens` y `lm_head`). Esta fusión consolida la parametrización en un archivo único libre de sobrecargas de enlace en tiempo de ejecución.

Mecánica de cuantización Q4_K_M A partir del binario consolidado en precisión de 16 bits, se aplica el algoritmo de cuantización por bloques Q4_K_M provisto por `llama.cpp` (Frantar et al., 2022; Gerganov, 2023). A diferencia de los métodos uniformes tradicionales donde todos los parámetros se reducen a la misma precisión, la estrategia de k-cuants (*k-quants*) adopta un esquema heterogéneo basado en la sensibilidad de cada estructura neuronal:

- **Tensores de auto-atención y activación (`q_proj`, `v_proj`, `gate_proj`):** Se comprimen a una representación de 4 bits por parámetro, organizados en super-bloques

que almacenan escalas y mínimos locales, permitiendo reconstruir la distribución de pesos con mínima pérdida de dispersión.

- **Capas críticas de alimentación hacia adelante y normalización:** Se retienen en una precisión mixta de 6 bits (*medium variant*), protegiendo las capas más propensas a generar desviaciones sintácticas o degradación en métricas de generación.

Impacto en huella de memoria y velocidad de procesamiento. Esta técnica de cuantización reduce sustancialmente el tamaño de los artefactos. Para el modelo Llama 3.2 1B, el peso en disco desciende a aproximadamente 770 MB, permitiendo que el binario sea descargado y alojado en memoria volátil sin activar paginación a disco (*swapping*). A su vez, `llama.cpp` está optimizado para ejecutarse de forma eficiente sobre procesadores estándar. Esto permitió alcanzar latencias promedio de apenas 330 a 420,ms por enunciado en la computadora de prueba, asegurando que la respuesta regrese antes de que se corte el ritmo del diálogo.

5.3.4. Manejo de contexto y memoria conversacional deslizante

El diálogo natural sostenido durante una videollamada rara vez se compone de enunciados aislados e independientes. Fenómenos lingüísticos como la elipsis de sujetos, el uso de pronombres (e.g., ESTO, ACA) o respuestas breves de confirmación exigen retener el hilo temático inmediato para que la interpretación semántica resulte coherente. Para dotar al sistema de persistencia temporal sin incurrir en degradación de rendimiento, se diseñó e implementó el componente `ConversationMemory`.

Modelado de turnos y estructura de datos. La memoria modela los intercambios a través de una abstracción orientada a objetos tipada denominada `Turn`, la cual registra los atributos esenciales de cada intervención (Gamma et al., 1994):

```
@dataclass
class Turn:
    role: str          # "signer" | "hearing"
    text: str          # Texto en español (signer) o transcripto (hearing)
    glosses: str       # Opcional, solo en turnos del señante
    timestamp: float
```

La diferenciación del rol emisor permite registrar tanto las traducciones generadas a partir de la LSA como los mensajes orales o escritos provenientes del interlocutor oyente. En los turnos del señante ("**signer**"), se preserva tanto la secuencia original de glosas como la oración final sintetizada en español, permitiendo que el modelo de lenguaje consulte ambos niveles de abstracción si es necesario.

Política de ventana deslizante por turnos totales. La gestión del historial conversacional se estructura bajo una política de ventana deslizante acotada a un límite de $N = 10$ turnos totales (`max_turns = 10`). La decisión de computar turnos totales en lugar de cupos rígidos individuales (e.g., 5 del señante y 5 del oyente) responde a la asimetría propia de los intercambios reales: en conversaciones fluidas es habitual que una de las partes realice múltiples intervenciones breves o aclaraciones consecutivas. Un esquema

basado en turnos totales asegura que el contexto inmediato más reciente permanezca siempre accesible en la memoria volátil.

Para garantizar seguridad frente a concurrencia (*thread-safety*), las operaciones de lectura, inserción y vaciado del búfer están protegidas mediante mecanismos de exclusión mutua (`threading.Lock` o `RLock`) (Pressman & Maxim, 2014). Esto previene condiciones de carrera (*race conditions*) entre el hilo de captura cinemática, el motor local ILSA y los procesos asíncronos que despachan peticiones hacia el servidor semántico.

Manejo del historial de conversación y contexto. Para darle continuidad al diálogo durante la videollamada, se incorporó el componente `ConversationMemory` con la idea de brindar contexto temático entre intervenciones consecutivas[cite: 1].

El modelo de lenguaje se entrenó originalmente sobre un conjunto de datos formado por pares de un solo turno (secuencias de glosas y su oración equivalente en español), ya que armar diálogos completos de múltiples turnos requería un esfuerzo de curación fuera del alcance de esta etapa[cite: 1].

Al momento de integrar la memoria, se consideró que pasarle mensajes previos podía alejar el formato de entrada de lo que el modelo vio durante el entrenamiento[cite: 1]. Por este motivo, se adoptaron las siguientes decisiones de diseño:

- **Historial configurable:** La inyección del contexto se configuró mediante una variable booleana (`USE_CONVERSATION_HISTORY`)[cite: 1]. Cuando la memoria se encuentra vacía o el mecanismo se deshabilita, el prompt entregado a la red coincide de forma exacta con el formato del entrenamiento supervisado, garantizando que el primer enunciado de cada conversación no sufra alteraciones[cite: 1].
- **Estructuración bajo formato de diálogo (*Chat Format*):** En lugar de concatenar el historial como texto plano, los turnos previos se inyectan en el prompt formateados como mensajes anteriores del diálogo: las intervenciones del oyente se asignan al rol de usuario (`role: "user"`) y las respuestas del señante al rol de asistente (`role: "assistant"`)[cite: 1]. De este modo, el modelo procesa el enunciado actual como la última respuesta a completar dentro de la conversación.
- **Reinicio del contexto:** Ante cambios de tema, cambios de interlocutor o al iniciar una nueva videollamada, el sistema expone el método `clear()`, accesible tanto a través del endpoint `REST POST /conversation/clear` del motor local ILSA como mediante atajos de teclado en la interfaz de usuario[cite: 1, 7]. Esto permite vaciar la ventana activa a cero sin comprometer el registro histórico de la sesión[cite: 1].

5.4. Integración en Google Meet y extensión de navegador

5.4.1. Desafíos del entorno Chrome Manifest V3: aislamiento offscreen y sandbox

Cuando intentamos llevar el pipeline de visión a una extensión de navegador, nos encontramos con las restricciones de seguridad que impone Chrome Manifest V3 (Google Chrome Developers, 2024). En esta versión de la plataforma, los scripts en segundo plano (*service workers*) no tienen acceso al DOM y prohíben evaluar código dinámico o cargar

scripts externos no empaquetados. Esto generó un conflicto directo con MediaPipe Holistic, ya que la librería utiliza WebAssembly y crea hilos auxiliares (*workers*) para procesar el video sin congelar la interfaz (Google, 2020).

En un contexto de extensión convencional, el navegador bloquea la inicialización de estos módulos auxiliares. Para superar esta limitación técnica y mantener la fluidez de la videollamada, diseñamos una arquitectura desacoplada en dos capas, cuya estructura e interacción general se ilustran en la Figura 5.4.

Página en segundo plano (*offscreen*) Dado que el *service worker* no puede manipular elementos visuales, creamos un documento sin ventana visible utilizando la API `chrome.offscreen` (Google Chrome Developers, 2024). Su función principal es recibir los cuadros de video en formato comprimido desde la videollamada y coordinar la comunicación con los demás componentes del sistema. Además, para reducir el consumo de CPU y memoria, bajamos la resolución de los cuadros de 320 a 240 píxeles de ancho.

Entorno aislado (*sandbox*) Para que MediaPipe pudiera ejecutarse sin infringir la política de seguridad del navegador (*Content Security Policy*), declaramos un marco aislado (*sandbox*) en el archivo de manifiesto (Google Chrome Developers, 2024). Esta página no tiene acceso directo a las APIs privilegiadas de la extensión ni a la red exterior. Sin embargo, cuenta con los permisos necesarios para instanciar el runtime de WebAssembly y cargar los modelos locales de pose y manos de MediaPipe Holistic (Google, 2020).

Canal de mensajería y flujo de datos El intercambio de información entre estos contextos se organizó mediante un puente de eventos:

1. La extensión toma la imagen de la cámara web y la envía al documento *offscreen* como imagen JPEG.
2. El documento *offscreen* reenvía el cuadro al *sandbox* a través de `postMessage`.
3. El entorno *sandbox* procesa la imagen con MediaPipe Holistic y extrae las 225 coordenadas tridimensionales de torso y manos (Google, 2020).
4. Los puntos calculados retornan al script *offscreen*, el cual analiza si hay manos activas, recorta la ventana temporal y despacha el vector cinemático al motor local ILSA mediante una petición HTTP POST (Ramírez, 2018).

Esta separación permitió que Google Meet corriera en su hilo principal sin caídas de cuadros, aislando por completo la carga computacional de la visión artificial dentro de procesos auxiliares del navegador.

5.4.2. Intercepción de `getUserMedia` y derivación de flujo vía `canvas.captureStream()`

Uno de los mayores desafíos al integrar el sistema en Google Meet fue el acceso a la cámara física. En los navegadores modernos, la API estándar `navigator.mediaDevices.getUserMedia` maneja la cámara como un recurso exclusivo: no permite que dos procesos consuman el mismo dispositivo al mismo tiempo sin bloquear el hardware.

En nuestro diseño necesitábamos dos flujos concurrentes: capturar los cuadros de video para extraer los puntos corporales en la extensión y, al mismo tiempo, alimentar

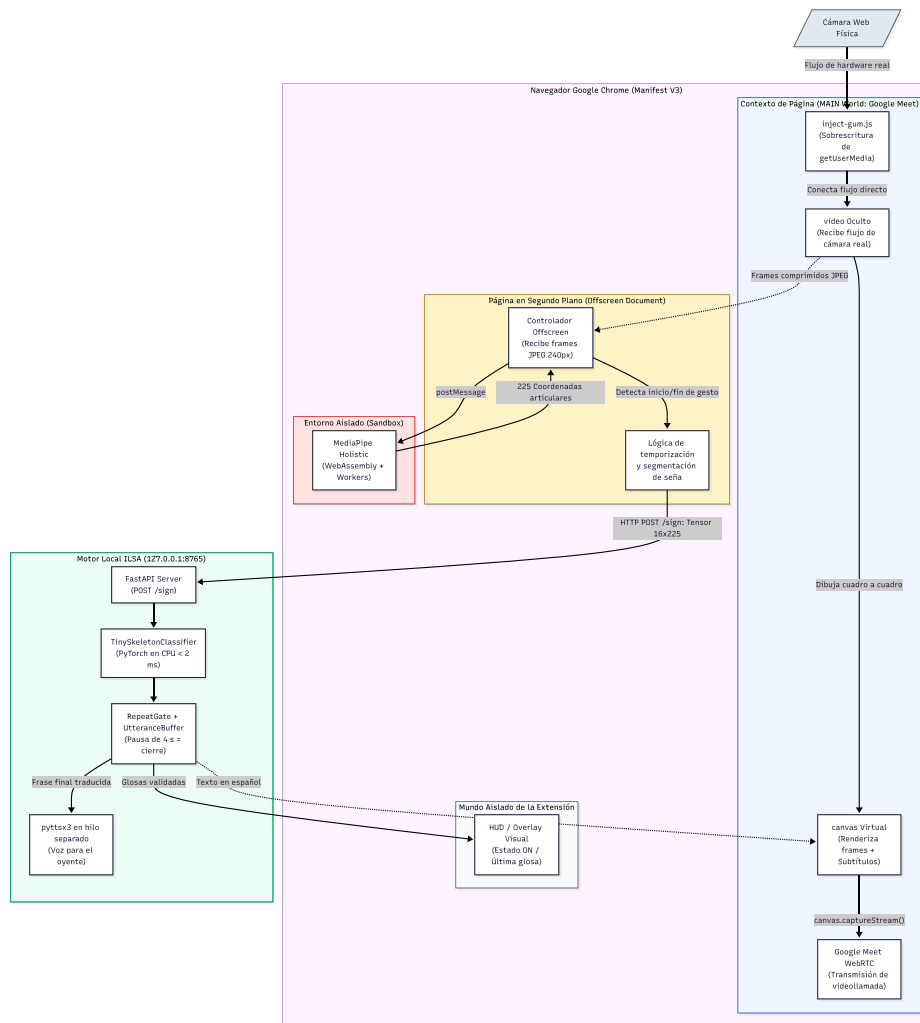


Figura 5.4: Arquitectura de integración en el navegador: intercepción de la cámara web, derivación de fotogramas al entorno aislado (*offscreen* y *sandbox*) y enlace con el motor local ILSA.

la transmisión nativa de la videollamada. Si MediaPipe tomaba el control directo de la cámara, Meet no podía transmitir video; y si Meet la iniciaba primero, el clasificador quedaba bloqueado.

Para resolver este problema sin alterar el funcionamiento interno de Google Meet, recurrimos a una técnica de intercepción dinámica en el contexto de ejecución principal de la página web (*MAIN world*):

Envoltorio de `getUserMedia` y desvío a video oculto Inyectamos un script (`inject-gum.js`) que sobrescribe la función `navigator.mediaDevices.getUserMedia` original. Cuando el motor ILSA está encendido y Meet solicita la cámara, el script intercepta la llamada, obtiene el flujo real del hardware y lo conecta a un elemento `<video>` invisible en segundo plano. De este modo, la plataforma de videollamada no toma el control del hardware directo.

Generación de flujo sintético con `canvas.captureStream()` Desde ese elemento de video oculto, tomamos los cuadros para derivarlos al flujo de MediaPipe y, en paralelo, los dibujamos cuadro a cuadro sobre un elemento `<canvas>` virtual. Luego, utilizando el método estandarizado `canvas.captureStream()` (W3C, 2024), sintetizamos un flujo de medios falso (*MediaStream*) que le entregamos a Google Meet como si fuera la cámara web original.

Ventajas del lienzo virtual y gestión de subtítulos Esta solución resolvió dos aspectos centrales:

1. **Evitó el bloqueo de hardware:** Ambos procesos comparten el mismo flujo visual sin errores de acceso ni conflictos de permisos.
2. **Superposición directa de texto en la señal:** Al controlar el dibujo del lienzo, pudimos pintar las oraciones en español traducidas directamente sobre la parte inferior del video. Así, el interlocutor oyente puede leer los subtítulos en su pantalla sin necesidad de tener instalada nuestra extensión en su navegador.

Para no entorpecer la visión de la llamada, implementamos un temporizador visual: el texto traducido permanece visible durante 8 segundos y desaparece automáticamente. Si llega un enunciado nuevo antes de ese plazo, el reloj se reinicia. Además, si el usuario apaga el motor ILSA, el script retira el interceptor y devuelve el control tradicional de la cámara a Google Meet de inmediato.

5.4.3. Motor Local ILSA (FastAPI)

Para coordinar las tareas locales sin sobrecargar el navegador web, desarrollamos el motor local ILSA (*Intérprete de Lengua de Señas Argentina*). Se trata de una aplicación de escritorio programada en Python que levanta un servidor HTTP local mediante FastAPI y Uvicorn en la dirección de bucle invertido `127.0.0.1:8765` (Ramírez, 2018).

Al diseñar este componente, aprendimos que desacoplar la lógica de clasificación y la síntesis de voz del entorno de Chrome simplifica el mantenimiento y evita problemas de rendimiento en la videollamada. El motor actúa como un orquestador que centraliza cuatro responsabilidades principales:

Recepción de secuencias y ejecución del clasificador A través del endpoint `POST /sign`, el motor recibe los paquetes JSON enviados por la extensión. Cada paquete contiene una secuencia normalizada de 16 fotogramas con 225 coordenadas cinemáticas. El motor convierte estos datos en un tensor de PyTorch con dimensiones (1, 16, 225) y ejecuta la inferencia local del modelo *TinySkeletonClassifier* en CPU en menos de 2 ms. El resultado se responde inmediatamente a la extensión para actualizar el indicador visual (HUD) en la pantalla del usuario.

Filtrado con RepeatGate y acumulación en UtteranceBuffer Cada predicción pasa por la compuerta `RepeatGate` (Gamma et al., 1994). Esta clase aplica reglas según el tipo de seña para filtrar rebotes no deseados: admite números repetidos, permite hasta dos letras iguales consecutivas y descarta repeticiones inmediatas en palabras generales. Las glosas aceptadas se van acumulando en el objeto `UtteranceBuffer`. Si pasan 4 segundos continuos sin actividad gestual, el búfer declara el cierre de la frase y dispara la petición al servidor semántico remoto para obtener la traducción en español.

Manejo de concurrencia y síntesis de voz (pyttsx3) Uno de los problemas técnicos que encontramos fue que la librería `pyttsx3` bloquea el hilo de ejecución mientras pronuncia una frase, además de depender de la arquitectura COM en Windows. Si corríamos la síntesis de voz en el hilo principal de FastAPI, el servidor dejaba de responder peticiones y la cámara se trababa. Para solucionarlo, implementamos una arquitectura concurrente con hilos desacoplados y colas seguras (`queue.Queue`) (Pressman & Maxim, 2014). El motor de voz corre en su propio proceso en segundo plano, reproduciendo el audio hacia el oyente sin retrasar la clasificación de nuevas señas.

Empaquetado y distribución ejecutable Para que el usuario final no tenga que instalar Python ni configurar entornos virtuales, empaquetamos el backend en un binario ejecutable independiente (`ILSA.exe`) utilizando PyInstaller. El ejecutable incluye una interfaz gráfica simple donde el usuario puede seleccionar el modo de trabajo (Sordo u Oyente), configurar la mano dominante y encender o apagar el servicio con un solo botón. Al iniciar la sesión, el motor descarga automáticamente la URL pública del servidor semántico desde los lanzamientos de GitHub, permitiendo que la extensión comience a operar en Google Meet de inmediato sin configuraciones manuales de red.

5.5. Representación gestual Inversa mediante avatar 3D

5.5.1. Controlador de animaciones esqueléticas por glosas

5.5.2. Mecanismo de contingencia: deletreo manual dactilológico

5.6. Diseño conceptual de la arquitectura escalable en la nube

La partición funcional adoptada establece que la carga computacional asociada al modelo de lenguaje (el módulo semántico) quede desacoplada de la computadora del

usuario y se atiende de manera remota. De este modo, la máquina cliente solo procesa el flujo de cámara y la inferencia gestual liviana, enviando únicamente texto plano (listas de glosas u oraciones) hacia el servidor exterior.

Sin embargo, para llevar esta solución a un entorno de uso masivo con múltiples videollamadas concurrentes, se requiere una infraestructura elástica capaz de procesar turnos simultáneos sin encolamientos ni demoras que corten el ritmo del diálogo. A continuación, se formaliza el diseño conceptual para entornos de producción y se detalla el esquema real con el que se validó el prototipo.

5.6.1. Especificación para entornos de producción concurrente

Para garantizar alta disponibilidad y responder en tiempos medios menores a los 450 ms por turno frente a múltiples usuarios concurrentes, se proyectó una arquitectura modular basada en microservicios contenerizados y auto-escalado elástico. La topología de recursos y el flujo de balanceo se describen en la Figura 5.5.

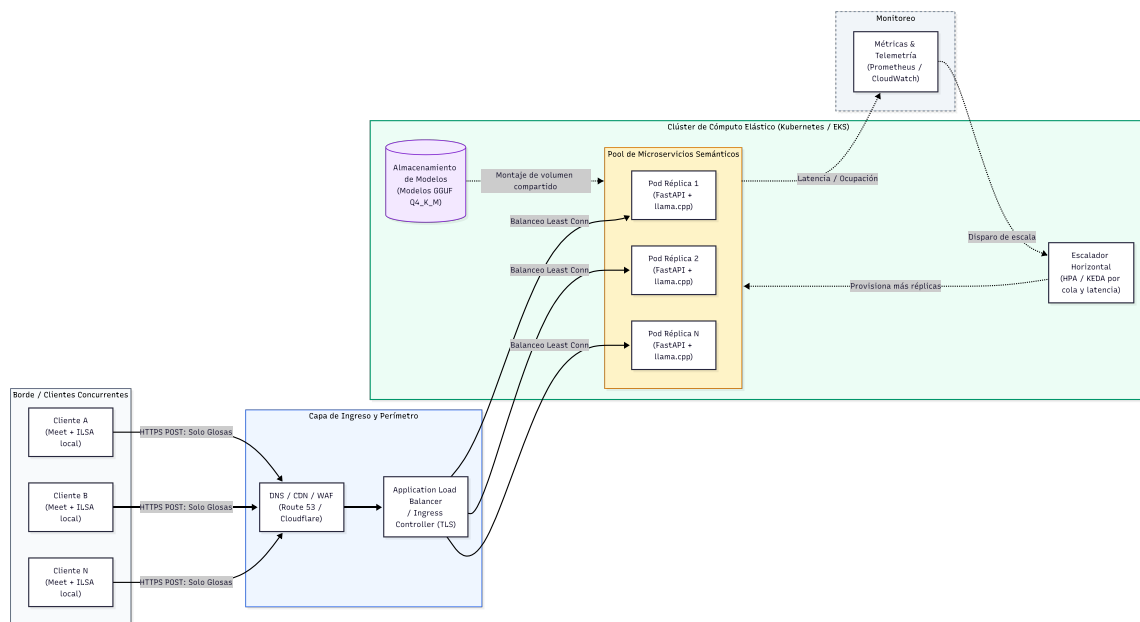


Figura 5.5: Diseño conceptual de la infraestructura en la nube: red perimetral, balanceo de carga y clúster elástico de inferencia semántica con escalado horizontal.

El diseño contempla tres capas principales de infraestructura:

Capa de acceso perimetral y balanceo: Las solicitudes HTTPS enviadas por las computadoras clientes son recibidas por una capa perimetral que incluye resolución DNS, certificados de seguridad TLS y filtrado de tráfico básico para prevenir saturaciones maliciosas. Detrás de esta capa opera un balanceador de carga (*API Gateway* o *Application Load Balancer*):

- Recibe y valida los esquemas JSON de las rutas `/translate` y `/hearing`.
- Distribuye las peticiones hacia las réplicas del microservicio semántico mediante un criterio de menor número de conexiones activas (*Least Connections*), evitando sobrecargar instancias que se encuentren resolviendo enunciados largos.

Clúster de cómputo y escalado elástico: El microservicio semántico (desarrollado con FastAPI y el motor de `llama.cpp`) se empaqueta en imágenes de contenedor Docker (Gerganov, 2023; Ramírez, 2018). Estos contenedores se despliegan sobre un clúster gestionado de Kubernetes (como AWS EKS o equivalente):

- **Almacenamiento compartido de modelos:** Los archivos binarios cuantizados en formato GGUF (Q4_K_M) se almacenan en un volumen persistente de solo lectura montado en cada instancia, evitando transferencias repetitivas de red al iniciar nuevos contenedores.
- **Auto-escalado por demanda:** Mediante un controlador de escalado horizontal (HPA), el clúster monitorea la longitud de la cola de peticiones y los tiempos de generación de respuesta. Cuando la cantidad de turnos simultáneos supera el umbral operativo de las réplicas activas, el orquestador provisiona automáticamente nuevos contenedores para absorber el pico de uso.
- **Eficiencia de hardware y costos:** Gracias a la cuantización a 4 bits (GGUF Q4_K_M), cada réplica requiere menos de 2 GB de memoria RAM y puede ejecutarse sobre procesadores estándar en la nube, prescindiendo de servidores dedicados con placa de video (GPU).

Monitoreo y telemetría: Se prevé un sistema de recopilación de métricas (como Prometheus y Grafana o AWS CloudWatch) para supervisar la latencia por oración, la memoria ocupada por contenedor y la tasa de errores de red, permitiendo calibrar con precisión las reglas de escalado.

5.6.2. Instancia de validación del prototipo mediante túnel seguro

El despliegue comercial continuo de un clúster elástico en proveedores como AWS o Google Cloud Platform implica costos periódicos de infraestructura que no resultaban viables dentro del marco de un Trabajo Final de Grado, tal como quedó establecido en la restricción RC-02 de requerimientos.

Para validar el funcionamiento del sistema desacoplado sin incurrir en costos operativos, se implementó una instancia remota de referencia utilizando un túnel seguro cifrado (*Cloudflare Tunnel* o *ngrok*).

Configuración de la instancia de prueba: El esquema operativo del prototipo se organizó de la siguiente forma:

1. **Servidor semántico desacoplado:** El servidor se levanta en una computadora separada del equipo donde corría la videollamada. Este proceso levanta el servidor FastAPI, cargando en memoria RAM los binarios cuantizados del traductor gestionados con `llama-server`.
2. **Túnel inverso cifrado:** A través del cliente de ngrok, se estableció un túnel TLS saliente que generó una URL pública con HTTPS (por ejemplo, `https://ilsa-semantic.trycloudfl`). Esto permitió exponer el servicio a Internet de forma segura sin requerir una IP pública fija ni configurar redirecciones de puertos en el rúter.

3. **Conexión del cliente local:** El motor ILSA en la computadora del usuario lee la URL remota mediante la variable de configuración `SEMANTIC_URL` o la consulta al iniciar la aplicación. Al detectar el fin de una frase de señas, despacha la lista de glosas por la red pública hacia dicho punto de acceso.

Validez técnica de las pruebas Este esquema reprodujo con exactitud el comportamiento que tendría el sistema en un entorno de producción real:

- Incorporó los retardos típicos de una red de telecomunicaciones residencial (latencias de subida y bajada, fluctuaciones de conexión y negociación TLS).
- Verificó que los contratos de datos y la serialización JSON funcionaran de manera consistente a través de Internet.
- Demostró que la computadora cliente de 8 GB de RAM se mantuvo liberada del consumo de memoria y CPU del modelo de lenguaje, garantizando que Google Meet y MediaPipe operaran a 30 FPS continuos sin caídas de rendimiento (Google, 2020).

De esta manera, se comprobó la viabilidad del modelo híbrido cliente-nube, dejando la aplicación preparada para migrar a una infraestructura en la nube simplemente actualizando la dirección URL del servicio semántico.

Capítulo 6

Experimentación, Resultados y Discusión

6.1. Corpus y metodología de adquisición de Datos de LSA

6.1.1. Definición y validación del vocabulario de 100 clases

6.1.2. Protocolo de grabación y curación del dataset propio

6.2. Evaluación empírica del clasificador gestual

6.2.1. Métricas offline vs. rendimiento en cámara real

6.2.2. Impacto de la resolución temporal: 16 vs. 32 fotogramas

6.2.3. Búsqueda automatizada de hiperparámetros con Optuna y Análisis de sobreajuste

6.2.4. Diagnóstico de confusiones mediante feature engineering y separabilidad

6.2.5. Impacto cuantitativo de la corrección de aspect ratio

6.2.6. Desempeño final del clasificador baseline en vivo

6.3. Evaluación de los módulos de traducción semántica

6.3.1. Benchmark comparativo de modelos SLM (Familias Qwen2.5, Llama 3.2 y SmolLM2)

6.3.2. Evaluación cuantitativa: BLEU, ROUGE-L, METEOR y exactitud

6.3.3. Análisis de simetría en el aprendizaje: traducción directa vs. estructuración inversa

6.3.4. Compromiso entre calidad lingüística y latencia de inferencia

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones

- 7.1.1. Cumplimiento de objetivos del proyecto
- 7.1.2. Aportes técnicos y de accesibilidad
- 7.1.3. Impacto social en el marco de la Ley 27.710

7.2. Trabajo futuro

- 7.2.1. Implementación física del clúster semántico escalable
- 7.2.2. Expansión del corpus multiseñante y mitigación de variabilidad
- 7.2.3. Incorporación de rasgos no manuales (expresiones faciales)
- 7.2.4. Generalización a nuevas plataformas de comunicación virtual

Bibliografía

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework, 2623-2631.
- Alibaba Cloud. (2024). Qwen2.5: A Family of Large Language Models.
- Allal, L. B., Lozhkov, A., Bakouch, E., von Werra, L., & Wolf, T. (2024). SmolLM2: When Small Goes Big.
- Barán, V. (2018). LSApp: Uniendo dos mundos.
- Blessit. (2024). Interpretación Lengua de Señas — Central de intérpretes.
- Coldewey, D. (2018). SignAll is slowly but surely building a sign language translation platform.
- Confederación Argentina de Sordos. (2024). Señario de términos y expresiones en Lengua de Señas Argentina.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- Frantar, E., Ashkboos, S., Torfi, A., & Alistarh, D. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323*.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Gerganov, G. (2023). *llama.cpp: Port of Facebook's LLaMA model in C/C++*. <https://github.com/ggerganov/llama.cpp>
- Google. (2020). MediaPipe: A Framework for Building Perception Pipelines.
- Google Chrome Developers. (2024). Overview of Manifest V3.
- Hand Talk. (2025). Acessibilidade digital em línguas de sinais.
- HETAH. (2024). Avatar traductor a lengua de señas.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *Proceedings of the International Conference on Learning Representations*.
- Hugging Face. (2024). Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX.
- INJU. (2024). Diccionario de Señas.
- Li, D., Rodriguez, C., Yu, X., & Li, H. (2020). Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison. *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, 1459-1469.
- Massone, M. I. (1993). El número y el género en la Lengua de Señas Argentina. *Signo & Seña*, 2, 73-99.
- Massone, M. I., & Martínez, R. A. (2012). *Curso de Lengua de Señas Argentina: Parte 1 y 2*. Cultura Sorda. <http://www.cultura-sorda.eu>

- Mindlin, I. (2021). *Reconocimiento de Lengua de Señas con redes neuronales recurrentes* [Tesis de grado]. Universidad Nacional de La Plata, Facultad de Informática. <http://sedici.unlp.edu.ar/handle/10915/129853>
- Ministerio de Justicia de la Nación. (2024). Características de la comunicación por video-llamadas — Línea 144.
- Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner’s Approach* (8th). McGraw-Hill.
- Ramírez, S. (2018). *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*. <https://github.com/fastapi/fastapi>
- República Argentina. (2020). Ley 27.710 — Lengua de Señas Argentina.
- Ronchetti, F., Quiroga, F., Estrebou, C., Lanzarini, L., & Rosete, A. (2016). LSA64: An Argentinian Sign Language Dataset. *XXII Congreso Argentino de Ciencias de la Computación (CACIC)*, 794-803.
- SignAll Technologies. (2026). About SignAll Technologies.
- Sorenson Communications. (2025). Sorenson Communications acquires OmniBridge and Hand Talk.
- Universidad Nacional de Mar del Plata, Facultad de Ingeniería. (2024). Protocolo de Trabajo Final — Ingeniería en Informática.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *arXiv preprint arXiv:1706.03762*.
- Verón, F., & Nigro, M. (2026). 2026_Proyecto_LSA.
- W3C. (2024). Media Capture from DOM Elements: `HTMLMediaElement.captureStream()`.
- Wu, Y., Lu, S., Gan, Z., Gan, C., Shen, Y., Chen, Y.-C., Smith, L., Ren, S., Carin, L., He, X., et al. (2020). Lite Transformer with Long-Short Range Attention. *arXiv preprint arXiv:2004.11886*.